Memory-lean density-fitted GPU4PySCF on P100, V100 and A100 GPUs: accuracy, speed and capacity against the original code and CPU PySCF

Qing Shao
Department of Chemical and Materials Engineering, University of Kentucky
qshao@uky.edu

October 2026

Abstract

We benchmark a set of changes to the density-fitted single-GPU path of GPU4PySCF (a bounded fitted-integral build, a new disk tier next to the existing host-memory tier, cuBLAS contraction fast paths, and a lower-memory analytic gradient) against the unmodified upstream code and against CPU PySCF 2.14.0. The campaign covers 127 cases (SCF energies and gradients from H_2O to Taxol with six functionals, Hessians, TDA and TDDFT, DF-MP2, CCSD, polarizabilities, shielding, solvation, dispersion, effective-core potentials, geometry optimizations and periodic systems) on P100 (12 and 16 GB), V100 (16 and 32 GB) and A100 (80 GB) GPUs, with two contraction engines (cuTENSOR and the CuPy fallback) and three storage tiers, in 14 648 audited records with accuracy recomputed against an independent CPU result.

Accuracy. Wherever a result qualified, original and new code agree with the CPU to within the declared gates (worst energy error 8.6×10^{-9} Eh against 10^{-8}; worst gradient 9.0×10^{-7} for the original and 8.5×10^{-8} Eh/Bohr for the new code against 10^{-6}). The original code fails the gradient gate on Taxol with hybrid functionals; wB97M-V and periodic MgO fail for both versions identically.

Speed. With cuTENSOR (V100, A100) the new code is as fast as the original (median original/new time ratio 1.00 on the A100 and 1.00 on the V100, per-case range 0.88–1.09 over both cards), with 3–8% slowdowns for some hybrid-functional cases. Without cuTENSOR (the P100 cards and a forced CuPy engine on V100 and A100) it is faster: 1.2–1.8× for intermediate-size hybrid-functional jobs and up to 2.39×, where the original code pays for `einsum`. Job types whose code is unchanged (Hessians, TDDFT, MP2, CCSD, properties, optimizations, periodic) show no difference.

Memory and capacity. The new code uses a median of 2.1× (up to 4.8×) less GPU memory for SCF energy-plus-gradient work with an exchange build. Combined with the host and disk tiers it lets a 12 GB P100 run valinomycin (1542 basis functions) and an 80 GB A100 a cluster of 139 water molecules (417 atoms, 3336 basis functions), against a 537-function limit on the small cards and 2400 functions on the A100 for the original code. Larger systems are limited by a gradient tensor of $N_{\mathrm{aux}}N_{\mathrm{occ}}^2$ elements, not by the fitted integrals.

Against the CPU. The GPU is a median of 46× (A100) and 36× (V100) faster than the best available CPU lane for cases that take the CPU at least 20 s, and up to 328× (A100) for finite-difference Hessians; a whole-node 48-thread CPU run was found to be slower than a 24-thread, single-BLAS-thread run in 120 of 127 cases (median 2.0× for cases above 20 s, up to 33× for a Hessian) because of nested BLAS threading, and both are reported.

Keywords: GPU computing, density functional theory, density fitting, PySCF, GPU4PySCF, benchmarking, quantum chemistry

1 Introduction

Graphics processing units (GPUs) have become a routine target for electronic-structure software. GPU4PySCF [3, 4] accelerates the density-functional and related methods of PySCF [1, 2] on NVIDIA GPUs, and its density-fitted (DF) single-GPU path [5, 6, 7] is the workhorse for medium-size molecules. Three practical limits recur when such code runs on the heterogeneous GPUs found in university clusters: the *memory* of the card (12–16 GB on older cards, 80 GB on the largest), the *contraction engine* available

(cuTENSOR may be missing or unsupported, in which case NumPy-style `einsum` contractions in CuPy [11] are used instead), and the need to know that a faster or smaller code gives *the same answer*.

This paper evaluates a set of changes to the DF single-GPU path of GPU4PySCF: a bounded fitted-integral build workspace with a disk tier alongside the existing host-memory tier, cuBLAS contraction fast paths for the Fock build, and a lower-memory analytic gradient. We compare it with the unmodified upstream code and with CPU PySCF on three GPU generations. We ask two questions:

1. Does the new code give *the same answer* as the original and as CPU PySCF across many job types, and is it *faster*?
2. Does it let *smaller-memory* GPUs treat *larger systems*, and what does this cost in time?

The measurements cover 127 benchmark cases on a university HPC cluster (the Lipscomb Compute Cluster, LCC) with P100 (12 and 16 GB), V100 (16 and 32 GB) and A100 (80 GB) cards, two contraction engines (cuTENSOR and the CuPy fallback) and three storage tiers for the fitted integrals. The CPU serves as an accuracy oracle and, in a separate lane, as a timing baseline; the emphasis is on the GPU code, and the CPU baseline is required only to be fair. The accuracy criteria were declared before any performance run, and an independent audit recomputes every statistic from the stored records.

The main results are as follows. The new code is numerically equivalent to the original within the declared gates. With cuTENSOR it is not faster than the original; without cuTENSOR it is up to 2.4 times faster. Its main benefit is memory: a median of 2.1 times (up to 4.8 times) less GPU memory, which lets a 12 GB card treat a system of 1542 basis functions and an 80 GB card one of 3336. We also report a caveat for CPU baselines: a whole-node 48-thread run with default BLAS threading is usually slower than a 24-thread run with one BLAS thread, so GPU-over-CPU speed-ups depend strongly on how the baseline is run.

Section 2 describes the code versions, hardware, cases, accuracy gates and timing protocol, including every deviation from the original plan. Section 3 describes the audit. Sections 4–9 give accuracy, speed, profiling and CPU-baseline results, and Section 10 the memory and capacity results. Section 11 discusses when the new code helps and what limits larger systems, Section 12 the limitations, and Section 13 the conclusions.

2 Experimental design

2.1 Code versions and the three-way comparison

Three implementations are compared on identical inputs.

CPU PySCF 2.14.0 [1, 2] with density fitting, the matching physical model for every case. It serves as the accuracy oracle and, in a separate lane, as the CPU timing baseline.

G0 (original GPU)
GPU4PySCF at upstream commit `9af748a`, with its Python sources unchanged.

G1 (new GPU)
The reviewed working tree of the `improve-single-gpu` branch at the time the campaign was frozen. Relative to G0 it changes the density-fitted (DF) fitted-integral storage (a bounded build workspace and a new disk tier; the host-memory tier already exists upstream), the Coulomb and exchange contractions used by the DF Fock build (cuBLAS GEMM, GEMV and SYRK fast paths that bypass `einsum` when the CuPy contraction engine is active), the DF analytic gradient (bounded temporary workspace, in-place application of the auxiliary metric, a copy-free auxiliary Gram matrix for the CuPy engine, explicit cuBLAS stream binding), a density-evaluation shortcut in the GGA XC quadrature, and the CMake architecture selection. Several further changes were made while preparing the campaign, all verified before the freeze (Section 2.7).

G0 and G1 share one native library stack, built once for compute capabilities 6.0, 7.0, 8.0 and 9.0 with CUDA 12.4 and LibXC 7.1.2 [10] compiled from source, so a single build serves every card. No native numerical kernel differs between G0 and G1; the comparison therefore isolates the Python-level algorithms and memory management. The contraction engine is an environment property and is recorded in every record: **cuTENSOR** (CuPy 13.4.1 [11] with cuTENSOR 2.2.0 [12], used wherever installed) or the **CuPy fallback** (`CONTRACT_ENGINE=cupy`, `einsum`-based). The fallback is the situation on hardware without a usable cuTENSOR, and it was the situation of an earlier single-GPU study of this branch on an NVIDIA GB10; here it is run deliberately on the V100 and A100 and is the only engine used on the P100 cards.

2.2 Hardware strata

Every card is a separate stratum and results are never pooled across strata. Every GPU job uses one GPU and eight CPU cores (16 for the capacity study, which also takes the whole node memory).

stratum	GPU	compute cap.	memory	host	engine
A100	A100 PCIe 80 GB	8.0	80 GB	Ice Lake, 56 cores, 256 GB	cuTENSOR, CuPy
V100	V100-SXM2 32 GB	7.0	32 GB	Skylake, 32 cores, 192 GB	cuTENSOR, CuPy
V100-16	V100-SXM2 16 GB	7.0	16 GB	Skylake, 32 cores, 192 GB	cuTENSOR (capacity only)
P100-16	P100-SXM2 16 GB	6.0	16 GB	Haswell, 16 cores, 128 GB	CuPy
P100-12	P100-PCIe 12 GB	6.0	12 GB	Skylake, 32 cores, 192 GB	CuPy
CPU	–	–	–	Cascade Lake, 48 cores, 192 GB	–

Table 1: Hardware strata of the cluster. All GPU nodes ran driver 550.54.14 (CUDA 12.4). The P100 cards have no tensor cores. The measured cuBLAS double-precision matrix-multiply ceiling (an 8192^3 product) is 16.3 TFLOPS on the A100 and 6.8 TFLOPS on the V100.

2.3 Benchmark cases and settings

The campaign registry contains **127 cases**: 60 *core* energy-plus-gradient cases (six systems from H_2O to Taxol with six methods; basis-set stress; open shells; charged and diffuse species; non-covalent dimers; two numerical-stress geometries), 56 *extension* cases covering other quantum-chemistry jobs, 6 direct (non-fitted) SCF cases, 4 periodic cases and one open-shell wB97X probe. The extension families are: additional DFT functionals and corrections (wB97M-V [13] with non-local correlation, effective-core potentials, D3BJ [14] and D4 [15] dispersion, IEF-PCM and SMD solvation; 18 cases), analytic Hessians (8), TDA and TDDFT excitation energies (12), DF-MP2 and unrestricted DF-MP2 (6), CCSD (2), static polarizabilities (3), NMR shielding (3) and geometry optimizations (4). All geometries are fixed inputs identified by SHA-256 hash; the small molecules are deterministic synthetic geometries, not optimized structures.

All DFT cases use a level-3 NWChem-pruned grid, SCF convergence 10^{-10} Eh (gradient 10^{-7}), a MINAO starting guess, FP64, def2-universal-JKfit [9] (with the def2 orbital bases [8]) as the fitting basis wherever density fitting is used (the CCSD, direct-SCF, polarizability and shielding cases use exact integrals) and, for gradients, auxiliary-basis and grid response. The same atomic starting density (hashed) is used for CPU and both GPU versions.

2.4 Accuracy criteria

Criteria were declared before any performance run. A GPU result *qualifies* only if it reproduces the independent CPU result: SCF energy within 10^{-8} Eh and every gradient component within 10^{-6} Eh/Bohr; Hessian within 10^{-5} Eh/Bohr^2; excitation energies within 10^{-6} Eh per matched root (state matching by transition-vector overlap); MP2 and CCSD energies within 10^{-8} Eh; polarizability within 10^{-5} a.u. and shielding within 0.1 ppm; periodic energies within 10^{-8} Eh. Results that fail a gate are kept and reported, and are excluded from equal-accuracy speed ratios; gates were not loosened after seeing results. All statistics are recomputed from the stored arrays by an independent audit (Section 3), which also verifies source and native-library hashes, settings, the requested job identity of every record, and that timing records carry no instrumentation.

2.5 Timing, memory and CPU protocol

GPU timing. Every observation is a fresh process; imports are outside the timer; the same tiny CUDA and BLAS initialization is run first; the timed region is the complete workflow (molecule and SCF construction, starting guess, fitted-integral construction, SCF, and the analytic gradient or other task). Original and new code are measured in a balanced, seeded AB/BA order, with five paired timing observations per case, extended to nine when either median absolute deviation exceeds 3% or the ratio lies between 0.95 and 1.05. A CuPy kernel cache that persists between processes is used (the first-process penalty is measured separately, Section 8).

Memory. One separate memory-mode process per case and code version wraps the CuPy allocator and records the peak of live requested GPU storage per phase. It excludes the CuPy pool overhead, library

workspaces and native allocations, and is not a total device-memory figure.

CPU. References for all cases were computed once with 24 threads. For the timing baseline each case is run in fresh processes on a whole 48-core node: first with 48 threads (the natural choice for a whole-node job), then with 24 threads and BLAS threads set to one (Section 9). Hessian cases are timed through the finite-difference oracle described below.

2.6 Capacity study

A ladder of systems of increasing size (vitamin C, tamoxifen, Taxol, valinomycin, three water clusters, glycine chains) is run with B3LYP/def2-SVP energy plus full-response gradient for each card, code version and storage tier (fitted integrals resident on the GPU, in pinned host memory, or in a file on disk). A column stops after two consecutive rungs fail for resource reasons. Build and staging budgets start at 4 GiB and are raised on demand to 1.2 times a rung's stated minimum (at most 35% of the card). Disk files are capped at 1.5 TB each and 4 TB in total.

2.7 Deviations from the plan and registry amendments

The following were fixed *before* the campaign freeze, each on measured evidence, and apply identically to CPU and GPU.

- **TDA tolerance** $10^{-9} \to 10^{-7}$ (six cases): PySCF's TDA solver does not report convergence at 10^{-9} even after 500 iterations; energies agree to six digits at 10^{-7}.
- **TDDFT tolerance** $10^{-9} \to 10^{-6}$ (six cases): the GPU solver reports convergence only for tolerances of 10^{-6} or looser, while CPU–GPU excitation energies differ by 4×10^{-9} Eh at every tolerance. As a consequence the GPU TDA calculations, which run at 10^{-7}, are much slower than the TDDFT ones and the two must not be compared with each other.
- **Polarizability and shielding** run with exact integrals (no density fitting) on both sides, because CPU PySCF raises `NotImplementedError` for these properties on density-fitted references.
- **Periodic cases** use FFT-based Coulomb (FFTDF) on both sides; they agree to 5.6×10^{-10} Eh. Gaussian density fitting with each library's default auxiliary basis differs by 1.8×10^{-4} Eh.
- **Dispersion** parameters are named explicitly (`d3bj:b3lyp`, `d4:b3lyp`) while the pinned VWN5 B3LYP coefficients are kept; PySCF's own name `b3lyp` is the VWN3 variant and differs by 0.037 Eh.
- **Hessian oracle.** The CPU analytic density-fitted Hessian ignores the grid response (3.7×10^{-4} Eh/Bohr^2 away from finite differences of the exact gradient), whereas the GPU Hessian matches finite differences to 1.7×10^{-6}. The oracle is therefore a central finite difference (step 10^{-3} Bohr, stability checked at 2×10^{-3}) of the CPU analytic gradient with auxiliary and grid response.
- **Geometry optimization** passes the GPU gradient to PySCF's optimizer as a gradient scanner.
- **Open-shell cases** (OH with B3LYP and PBE, NO with PBE, and the OH wB97X probe) do not converge on CPU from the MINAO guess in 100 cycles and are kept as terminal *non-converged* outcomes; no tuned guess was substituted.

Further changes to G1 were made after the initial single-GPU studies and before the freeze. Two came from memory profiling of the DF gradient: the auxiliary metric is applied in place (one $N_{\mathrm{aux}} N_{\mathrm{occ}}^2$ tensor instead of two), and a copy-free auxiliary Gram matrix replaces an `einsum` temporary of the same size in the CuPy engine (found with a labelled allocator trace on the P100). Two came from an independent code review of the working-tree diff: the gradient workspace budget falls back to the true minimum when free memory allows it, and the raw cuBLAS handle is bound to the current stream (and restored). New unit tests cover the fast contraction paths (58 DF and gradient tests pass in both engines). A candidate (caching pinned staging buffers for the disk tier) was tried and reverted after an alternating A/B/A/B measurement showed no gain. Review findings left unchanged are listed in Section 12.

3 Audit and coverage of the records

All numbers in this report come from 14 648 immutable job records written by the campaign (six GPU strata plus the CPU timing lane), each of which was audited by a program independent of the one that

produced it. The audit recomputes every accuracy statistic from the stored arrays and compares it with the stored one, checks the SHA-256 identity of the Python sources, native libraries, helper files, inputs and initial densities against the frozen campaign, verifies that every record's request (case, version, mode, attempt, scientific signature) matches its directory and result, that no timing record carries allocation or profiling instrumentation, that each timing slot is unique, and that every GPU result points to an audited CPU reference. Per stratum the audited record counts are 2384 (A100), 2399 (V100), 2184 (P100-16), 2193 (P100-12), 2247 (A100, CuPy engine) and 2265 (V100, CuPy engine).

Table 2 and Figure 1 give the outcome of the memory-mode run of each (case, code version) pair. About 87–88% of outcomes on the A100 and V100 are *qualified*: the result reproduces the independent CPU result within the declared gate. The rest are explained in Section 4: non-converged cases, accuracy failures (the original code on Taxol, and two job types that fail identically in both code versions) and, on the two P100 cards, genuine memory limits.

stratum	outcomes	qualified	accuracy failed	non-converged	resource limited	infrastructure	audited records
A100	248	223	11	14	0	0	2384
V100	248	224	11	13	0	0	2399
A100 CuPy	248	222	12	14	0	0	2247
V100 CuPy	250	224	10	13	0	3	2265
P100-16	248	212	8	14	9	5	2184
P100-12	248	215	8	14	6	5	2193

Table 2: Outcomes of the memory-mode run of each (case, code version) pair on each stratum, and the number of audited records. 'Infrastructure' on the P100 cards includes plain CUDA out-of-memory errors that the runner labelled generically (see text).

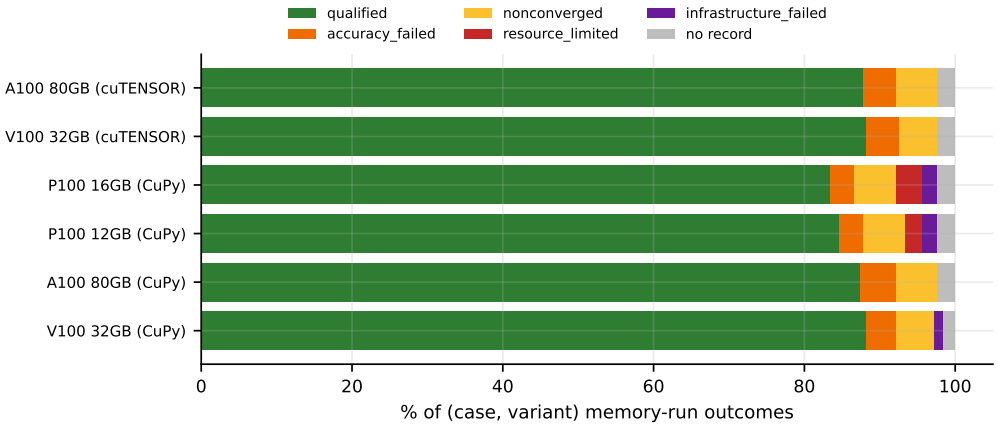


Figure 1: Outcome of the memory-mode run of each case and code version on each stratum. "No record" are the four cases whose CPU reference did not converge and the pre-launch rejections noted in Section 12.

4 Accuracy

Qualified results. Among all qualified GPU results the largest deviation from CPU PySCF, for both code versions, was 8.6×10^{-9} Eh in the SCF energy (the gate is 10^{-8}), 9.0×10^{-7} Eh/Bohr in a gradient component (gate 10^{-6}), 9.6×10^{-7} Eh/Bohr^2 in a Hessian element (gate 10^{-5}), 2.1×10^{-8} Eh in an excitation energy (gate 10^{-6}), 2.8×10^{-9} Eh in a correlation energy, 2.3×10^{-7} in a polarizability component (gate 10^{-5}) and 5.3×10^{-5} ppm in a shielding. Figure 2 shows the distribution. The new code is not less accurate than the original: its worst qualified gradient error (8.5×10^{-8} Eh/Bohr, for NO with B3LYP on the P100-16) is an order of magnitude below the original's worst (9.0×10^{-7}, tamoxifen B3LYP/def2-TZVP on the V100 with the CuPy engine), and over the 1314 pairs of error values compared the new code's error is smaller in 606, larger in 466 and equal in 242 (median ratio 1.00), so the two versions differ by no more than noise.

Failures. Table 3 lists every group of results that did not qualify. Two of them affect *both* code versions identically on every card, so they are properties of the shared native stack or of the GPU methods and

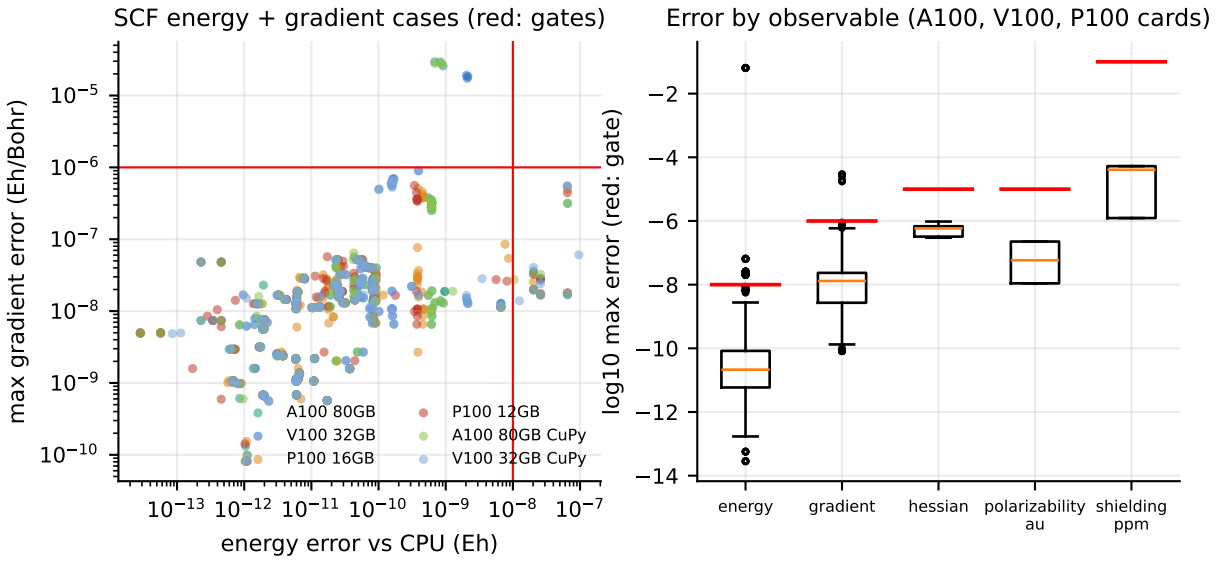


Figure 2: Accuracy against the independent CPU result. Left: SCF energy and maximum gradient error of every energy-plus-gradient case that was run, for both code versions and all strata (red lines: gates; points beyond them are the failures listed in Table 3). Right: log of the maximum error per observable over qualified results (red: gate).

not of the changes under test: wB97M-V with its non-local correlation reproduces the CPU energy only to 2–6 $\times 10^{-8}$ Eh for benzene, vitamin C and tamoxifen (it passes for H_2O at 6.6×10^{-9}), and the periodic MgO case differs by 6.4×10^{-2} Eh while periodic silicon agrees to 5×10^{-10} Eh. The wB97M-V offset grows with system size and is identical for the original and new code to the precision recorded; its cause was not investigated here.

group	affects	observation
Taxol HF, B3LYP, wB97X gradients	original only	maximum gradient error about 2×10^{-5} Eh/Bohr against the gate 10^{-6}; the new code passes (about 5×10^{-9}). No equal-accuracy ratio exists for these cases.
wB97M-V (benzene, vitamin C, tamoxifen)	both	energy error 2.6×10^{-8}, 2.0×10^{-8}, 6.4×10^{-8} Eh (gate 10^{-8}); gradients within the gate.
periodic MgO, Γ point	both	energy differs by 6.4×10^{-2} Eh on every card.
OH (B3LYP, PBE), NO (PBE), OH wB97X probe	all	CPU reference did not converge in 100 cycles from the MINAO guess; excluded.
benzene TDA and TDDFT	both	two of five roots flagged non-converged by the GPU solver at the declared tolerance; excitation energies agree with CPU to 1.2×10^{-9} Eh (diagnostic).
NO, B3LYP, 48-thread CPU repetitions	CPU lane	two of three repetitions differ from the 24-thread reference by 1.1 and 1.3 $\times 10^{-8}$ Eh; an ill-conditioned case.
CCSD on Pascal (P100)	both	native kernel launch error `fill_int4c2e`; cause not diagnosed.
Taxol and tamoxifen/TZVP hybrids on P100	both	GPU memory exhausted (Section 10).

Table 3: Groups of results that did not qualify. "Both" means the original and the new code behave identically.

5 Speed on the datacenter GPUs with cuTENSOR

With cuTENSOR installed (the default on the V100 and A100), the new code is *not faster* than the original. Over the 110 cases for which both versions qualified, the median ratio of original to new time is 1.00 on the A100 (range 0.88–1.04) and 1.00 on the V100 (range 0.91–1.09); restricting to the 32 (A100) and 31 (V100)

cases where the new code takes at least five seconds, so that start-up costs matter less, gives medians of 1.00 and 1.00 (Figure 4). Every job family is within ±3% of unity on both cards (Table 4): SCF geometric mean 1.00, Hessians 1.00–1.01, TDA/TDDFT 1.00, DF-MP2 and CCSD 0.97–1.01, properties 1.00–1.01, geometry optimizations 1.00–1.01, solvated calculations 0.99–1.00.

family	cases	A100	V100	A100 CuPy	V100 CuPy	P100-16	P100-12
correlation	8	0.97 (8)	1.00 (8)	1.01 (8)	1.01 (8)	1.02 (6)	0.96 (6)
hessian	8	1.00 (8)	1.01 (8)	1.03 (8)	1.02 (8)	1.02 (8)	1.02 (8)
optimization	4	1.00 (4)	1.01 (4)	1.02 (4)	1.02 (4)	1.01 (4)	1.03 (4)
pbc	4	0.99 (3)	1.02 (3)	1.02 (3)	1.02 (3)	1.02 (3)	1.01 (3)
properties	6	1.00 (6)	1.01 (6)	1.00 (6)	1.00 (6)	1.00 (6)	0.99 (6)
scf	77	1.00 (67)	1.00 (67)	1.07 (66)	1.05 (67)	1.06 (63)	1.09 (66)
solvent	8	0.99 (8)	1.00 (8)	1.05 (8)	1.05 (8)	1.07 (8)	1.14 (8)
td	12	1.00 (6)	1.00 (6)	1.01 (6)	1.01 (6)	1.02 (6)	1.02 (6)

Table 4: Geometric mean of the time ratio original/new per job family (number of cases in brackets); only cases where both versions passed the accuracy gate are included. Values above 1 mean the new code is faster.

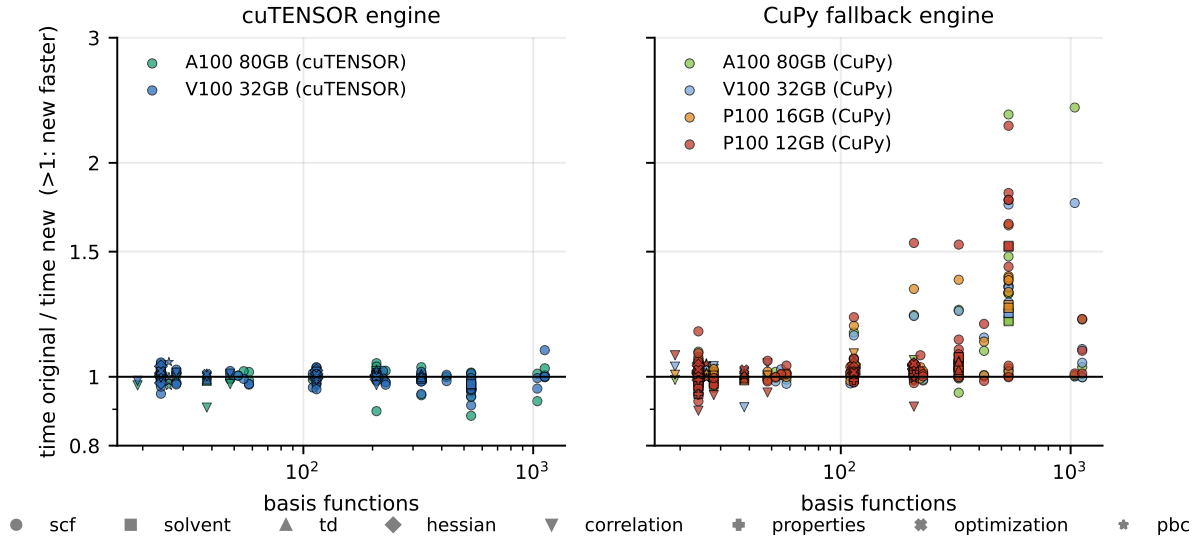


Figure 3: Time of the original code divided by time of the new code for every case in which both versions qualified, against the number of basis functions (marker shape = job family). Left: cuTENSOR engine. Right: CuPy fallback engine.

The larger cases (Table 5) show where the small differences occur. With the pure functionals LDA, PBE and TPSS the ratios are 1.00–1.09 (Taxol PBE: 1.03 on the A100 and 1.09 on the V100). For hybrid functionals the new code is slightly *slower*: tamoxifen/B3LYP 0.96, tamoxifen/wB97X 0.94, tamoxifen/B3LYP in the larger def2-TZVP basis 0.92 (A100) and 0.96 (V100), and the PCM and SMD solvation cases 0.97. These differences of 3–8% are small but reproducible between cards; they correspond to the larger gradient cost of the memory-saving gradient (Section 7).

6 Speed without cuTENSOR: the CuPy fallback and the P100 cards

The picture changes when contractions run through the CuPy `einsum` fallback, which is the only engine on the P100 cards and was forced on the V100 and A100 for a complete second stratum. The new code's Fock-build and gradient contractions use cuBLAS GEMM, GEMV and SYRK directly on this path, whereas the original code calls `einsum` and pays its repacking and temporaries. Over all cases the median original/new ratio is then 1.01 (A100), 1.01 (V100), 1.01 (P100-16) and 1.02 (P100-12); over the cases where the new code takes at least five seconds it is 1.02, 1.03, 1.04 and 1.05, and the SCF family geometric mean is 1.05–1.09

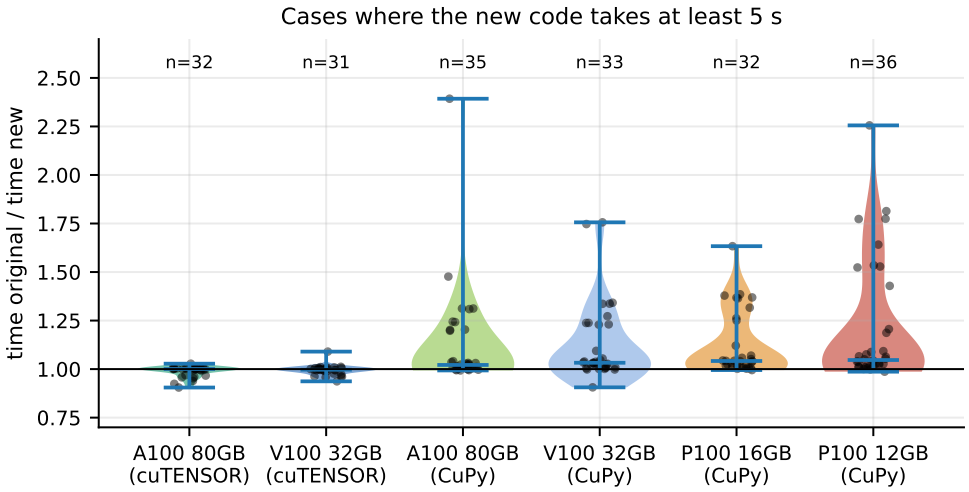


Figure 4: Distribution of the original/new time ratio over cases where the new code takes at least five seconds, per stratum (dots: individual cases).

case	AOs	A100	V100
`vitamin_c-b3lyp-svp`	208	2.34 / 1.04	2.18 / 1.01
`inosine-b3lyp-svp`	326	3.69 / 1.01	3.98 / 0.99
`tamoxifen-lda-svp`	537	4.67 / 1.00	4.81 / 0.99
`tamoxifen-pbe-svp`	537	5.34 / 1.01	5.70 / 1.01
`tamoxifen-tpss-svp`	537	8.05 / 1.01	8.72 / 1.00
`tamoxifen-b3lyp-svp`	537	6.49 / 0.96	7.80 / 0.96
`tamoxifen-wb97x-svp`	537	8.86 / 0.94	10.8 / 0.94
`tamoxifen-pbe-tzvp`	1042	8.48 / 1.01	10.8 / 1.00
`tamoxifen-b3lyp-tzvp`	1042	12.8 / 0.92	18.2 / 0.96
`taxol-lda-svp`	1123	14.5 / 1.01	18.3 / 1.00
`taxol-pbe-svp`	1123	18.0 / 1.03	24.2 / 1.09
`taxol-tpss-svp`	1123	26.4 / 1.00	38.1 / 1.00
`taxol-hf-svp`	1123	25.3 / (G0 acc.)	47.5 / (G0 acc.)
`taxol-b3lyp-svp`	1123	33.2 / (G0 acc.)	58.8 / (G0 acc.)
`taxol-wb97x-svp`	1123	49.1 / (G0 acc.)	85.1 / (G0 acc.)
`tamoxifen-b3lyp-iefpcm`	537	8.57 / 0.97	10.4 / 0.97
`tamoxifen-b3lyp-smd`	537	8.52 / 0.97	10.4 / 0.98
`tamoxifen-wb97mv`	537	–	–
`vitamin_c-b3lyp-hessian`	208	20.1 / 1.00	20.9 / 0.99
`inosine-b3lyp-tddft`	326	8.99 / 1.00	9.87 / 1.01

Table 5: Larger cases, cuTENSOR engine: new-code time in seconds / ratio original-over-new. 'mem.' = did not fit in GPU memory or was refused by the memory forecast; 'nc' = reference or GPU solver did not converge; '(G0 acc.)' = the original code fails the CPU accuracy gate, so no equal-accuracy ratio exists.

(Table 4). The gain is concentrated in hybrid-functional cases of intermediate size (Table 6): tamoxifen with B3LYP 1.31 (A100) to 1.77 (P100-12), with wB97X 1.27–1.64, with PCM or SMD 1.20–1.53, and tamoxifen/B3LYP in def2-TZVP 2.39 on the A100 and 1.76 on the V100; the pure-functional cases gain little (PBE 1.00–1.04) and Taxol with TPSS gains 1.05–1.21. The largest ratio observed in any stratum is 2.39 (A100, CuPy engine).

case	AOs	A100 CuPy	V100 CuPy	P100-16	P100-12
`vitamin_c-b3lyp-svp`	208	2.27 / 1.03	2.30 / 1.02	3.21 / 1.02	3.49 / 1.03
`inosine-b3lyp-svp`	326	4.00 / 1.02	4.39 / 1.05	6.55 / 1.05	7.01 / 1.09
`tamoxifen-lda-svp`	537	4.70 / 1.01	4.75 / 1.00	7.28 / 1.00	7.61 / 1.00
`tamoxifen-pbe-svp`	537	5.43 / 1.03	5.71 / 1.02	8.75 / 1.02	9.46 / 1.04
`tamoxifen-tpss-svp`	537	9.04 / 1.24	9.00 / 1.24	13.7 / 1.36	14.2 / 1.43
`tamoxifen-b3lyp-svp`	537	6.67 / 1.31	8.64 / 1.34	12.0 / 1.39	13.4 / 1.77
`tamoxifen-wb97x-svp`	537	9.20 / 1.48	12.2 / 1.27	16.8 / 1.32	19.2 / 1.64
`tamoxifen-pbe-tzvp`	1042	8.55 / 1.00	10.9 / 1.00	17.7 / 1.01	20.4 / 1.01
`tamoxifen-b3lyp-tzvp`	1042	12.7 / 2.39	18.5 / 1.76	–	–
`taxol-lda-svp`	1123	14.6 / 1.00	18.5 / 1.00	–	34.5 / 1.01
`taxol-pbe-svp`	1123	17.9 / 1.03	24.3 / 1.09	–	46.1 / 1.09
`taxol-tpss-svp`	1123	27.6 / 1.20	39.3 / 1.05	–	69.8 / 1.21
`taxol-hf-svp`	1123	24.6 / (G0 acc.)	–	mem.	mem.
`taxol-b3lyp-svp`	1123	32.8 / (G0 acc.)	–	mem.	mem.
`taxol-wb97x-svp`	1123	48.5 / (G0 acc.)	–	mem.	mem.
`tamoxifen-b3lyp-iefpcm`	537	8.83 / 1.20	11.4 / 1.23	16.1 / 1.26	18.0 / 1.52
`tamoxifen-b3lyp-smd`	537	8.69 / 1.20	11.2 / 1.23	16.0 / 1.25	18.0 / 1.53
`tamoxifen-wb97mv`	537	–	–	–	–
`vitamin_c-b3lyp-hessian`	208	26.1 / 1.04	25.3 / 1.04	37.8 / 1.04	36.3 / 1.04
`inosine-b3lyp-tddft`	326	10.2 / 1.01	11.6 / 1.03	17.1 / 1.07	21.4 / 1.05

Table 6: Larger cases, CuPy fallback engine: new-code time in seconds / ratio original-over-new. 'mem.' = did not fit in GPU memory or was refused by the memory forecast; 'nc' = reference or GPU solver did not converge; '(G0 acc.)' = the original code fails the CPU accuracy gate, so no equal-accuracy ratio exists.

Figure 5 shows why. For tamoxifen with B3LYP the original code takes 8.3 s with cuTENSOR on the A100 and 10.4 s with the CuPy engine (7.9 and 14.3 s on the V100), while the new code takes 8.7 and 8.8 s on the A100 (8.3 and 9.2 s on the V100): the new code makes the fallback engine as fast as cuTENSOR, but with cuTENSOR installed its fast paths are bypassed and it gains nothing. Put differently, the changes remove a penalty that only the fallback engine pays. This is the regime of the earlier GB10 study (Appendix B), where speed-ups of 1.47 (tamoxifen) and 1.95 (Taxol) were measured without cuTENSOR; the cluster data reproduce the direction and show that it depends on the engine, but with smaller factors (1.2–1.8 for tamoxifen hybrids).

7 Where the time goes and what did not help

Stage profile. Figure 6 splits the Taxol B3LYP/def2-SVP workflow (20 SCF cycles) into stages. On the A100 the SCF takes 21.9 s (new code), of which the Fock build is 13.9 s (including 3.5 s to build the fitted integrals once), the XC quadrature 7.1 s and the diagonalization 0.4 s; the gradient takes 13.0 s, 76% of it in the DF Coulomb and exchange derivatives. On the V100 the Fock build dominates even more (32.7 of 42.2 s SCF), as expected from its lower double-precision rate. The grid itself is built in about 0.1 s.

The matrix products are near the hardware ceiling. A micro-benchmark of the exchange-build contractions at the real Taxol shapes (Figure 7) shows both the cuTENSOR path and the cuBLAS GEMM/SYRK path running at 14–15 TFLOPS on the A100 (the measured cuBLAS ceiling is 16.3) and 5.8 TFLOPS on the V100 (ceiling 6.8), i.e. 85–90% of peak; the GEMM/SYRK path is 0–13% faster per cycle. There is little left to gain from tuning these products.

Experiments that did not pay.

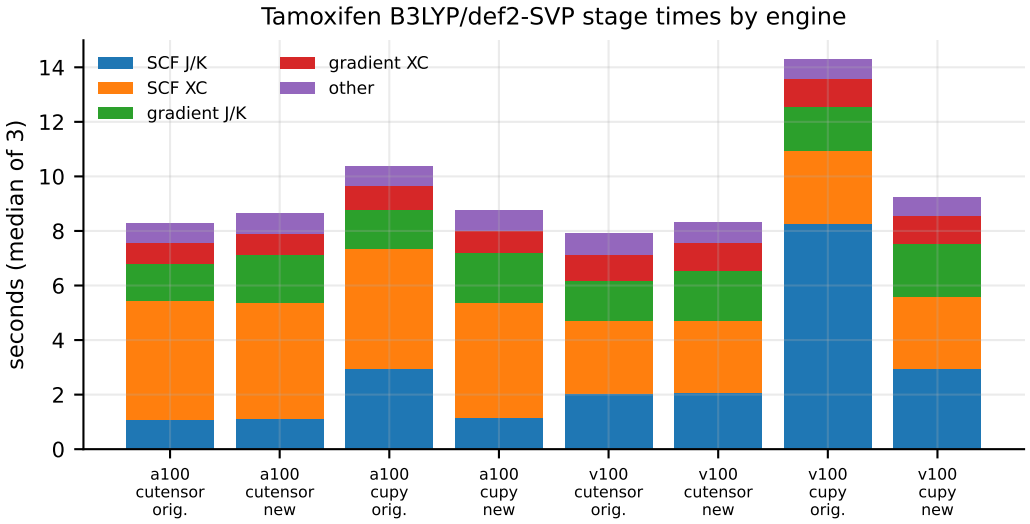


Figure 5: Stage times of tamoxifen B3LYP/def2-SVP (median of three, synchronized timers) for the original and new code with each engine. With cuTENSOR both versions take about 8–9 s; with the CuPy fallback the original code is slower (10.4 s on the A100, 14.3 s on the V100) and the new code is unchanged.

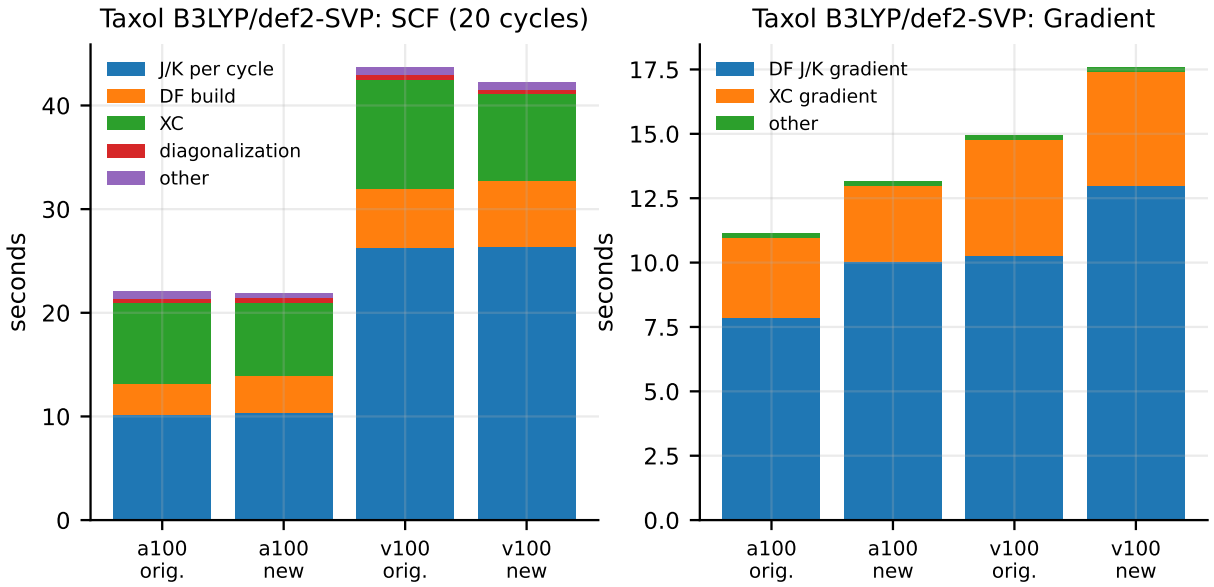


Figure 6: Stage times of Taxol B3LYP/def2-SVP on the A100 and V100 with cuTENSOR, original (orig.) and new code. The new code's gradient is 2–3 s slower, the price of its much smaller gradient memory.

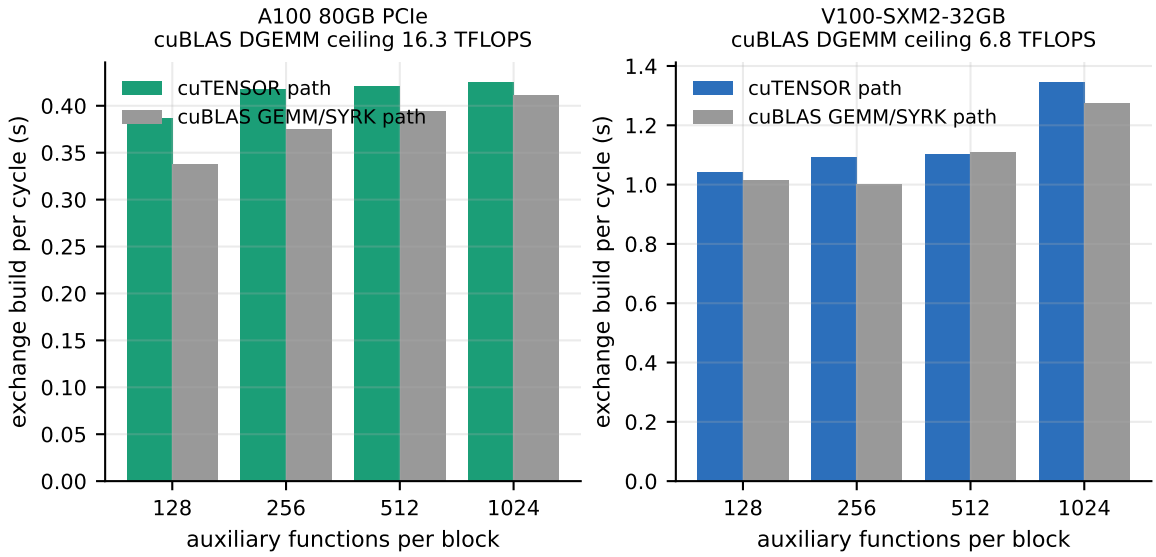


Figure 7: Estimated exchange-build time per SCF cycle for Taxol (one block of auxiliary functions timed, scaled to the full auxiliary basis) for the cuTENSOR contraction and the cuBLAS GEMM/SYRK path.

- *Gradient workspace cap.* Sweeping the fixed workspace budget of the new gradient (1, 2, 4, 8, 16 GiB and unlimited) changed the Taxol gradient time by less than 1% above 2 GiB (13.0–13.1 s on the A100, 17.5–17.6 s on the V100), and by 8% at 1 GiB. The planner already chooses the same batch size at the default.
- *SCF cycle count.* Table 7 and Figure 8: other starting guesses are equal (atomic densities) or much worse (Hückel, VSAP); a DIIS space of 12 saves one or two cycles; a two-stage scheme (coarse grid first) reduces the full-quality cycles from 20 to 12 for Taxol but its cheap pre-cycles cost as much (22.0 s against 22.2 s). Second-order SCF failed in all runs with a CuPy-to-NumPy conversion error.
- *Cached staging buffers for the disk tier* (a code-review suggestion): an alternating A/B/A/B comparison gave 265, 239 s (without) against 254, 280 s (with); the run-to-run spread exceeds any difference and the change was reverted.

strategy	vitamin C	tamoxifen	Taxol
MINAO (baseline)	18 / 3.8	18 / 6.0	20 / 22.2
superposition of atomic densities	18 / 4.0	19 / 6.0	21 / 22.5
extended Hückel	24 / 3.9	28 / 7.3	39 / 36.7
superposition of atomic potentials	23 / 5.7	27 / 24.7	39 / 153.2
MINAO, DIIS space 12	16 / 3.5	17 / 5.7	19 / 21.6
two-stage (coarse grid first)	11 / 3.5	12 / 5.6	12 / 22.0
second-order (SOSCF)	failed	failed	failed

Table 7: SCF cycles at full quality / total SCF seconds on the A100 for different starting guesses and convergence strategies (all reach the same energy within 10^{-8} Eh).

8 First-process (cold start) cost

CuPy compiles some kernels at run time. The first process on a node with an empty kernel cache therefore takes much longer than a repeat (Table 8 and Figure 9): the H_2O B3LYP workflow takes 12.6–23.0 s the first time and 0.9–1.9 s with a populated cache, an overhead of 11–22 s that is the same for the original and the new code and is largest on the P100. The principal timings of this report use a persistent cache; the figure matters mainly for short jobs and for users whose cache is not preserved between runs.

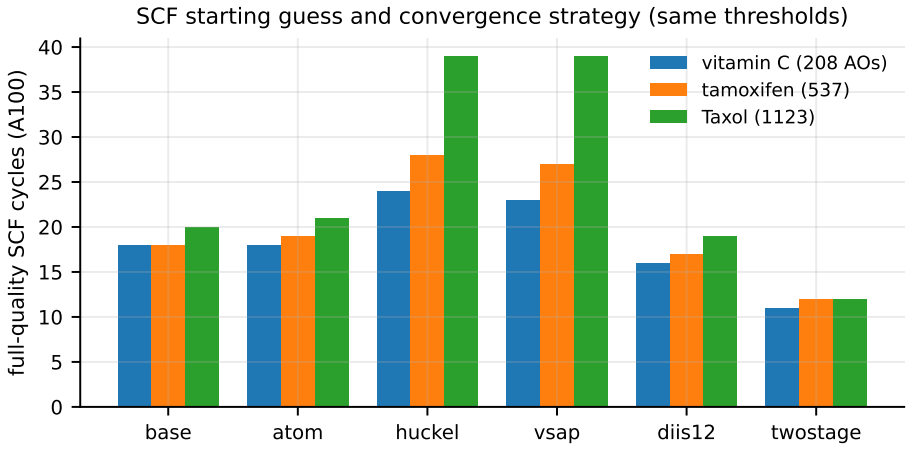


Figure 8: SCF cycles at full quality for different starting guesses and convergence strategies on the A100.

card	engine	code	H_2O cold (s)	H_2O warm (s)	tamoxifen cold (s)	tamoxifen warm (s)	first-process overhead (s)
A100	cupy	original	17.6	1.90	11.3	10.8	15.7
	cupy	new	19.2	1.84	8.67	8.58	17.3
	cutensor	original	19.6	1.94	8.36	8.14	17.7
	cutensor	new	17.5	1.81	8.37	8.55	15.7
V100	cupy	original	14.5	1.03	12.2	11.6	13.5
	cupy	new	13.4	0.91	9.19	8.47	12.5
	cutensor	original	13.0	0.86	7.56	7.37	12.2
	cutensor	new	12.6	0.89	7.63	7.59	11.7
P100-16	cupy	original	17.8	1.21	17.2	16.7	16.6
	cupy	new	17.6	1.20	12.0	12.1	16.4
P100-12	cupy	original	23.0	1.35	28.3	25.5	21.7
	cupy	new	22.4	1.32	14.2	14.2	21.1

Table 8: Cold start: the first process with an empty CuPy kernel cache against a repeat with the cache populated (workflow time inside the process, imports excluded). Only the first process of each pass is truly cold, so the first-process overhead is the H_2O cold time minus its warm time; the later, larger cases of the cold pass reuse kernels compiled by H_2O.

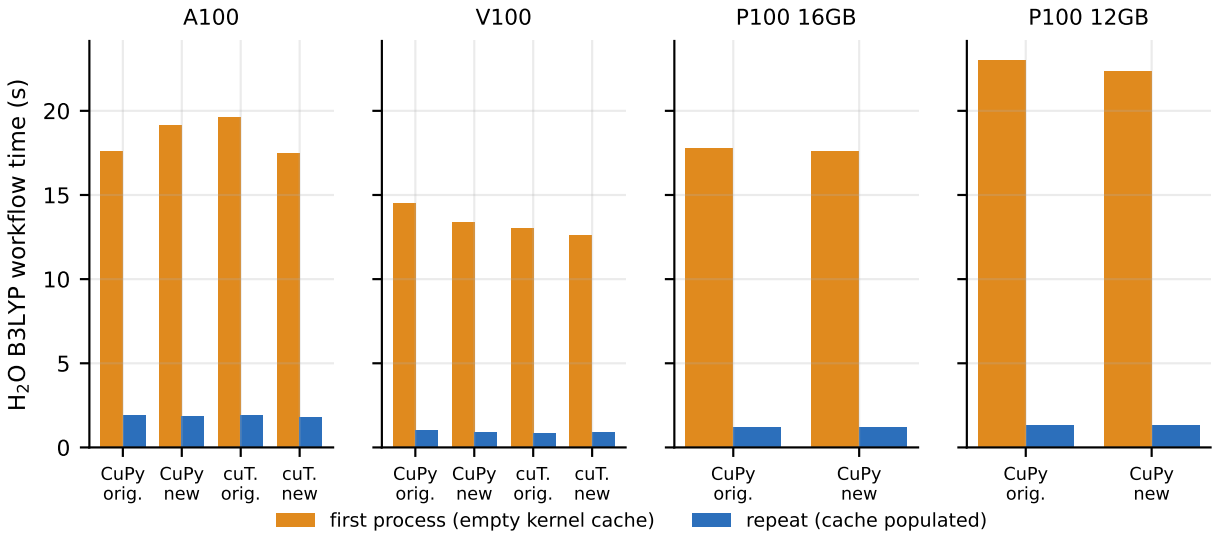


Figure 9: H_2O B3LYP/def2-SVP workflow time in the first process after the kernel cache was emptied and in a repeat.

9 CPU baseline and the GPU speed-up over the CPU

The study optimizes the GPU code, so the CPU is required only to be a fair baseline. The first CPU timing lane used a whole 48-core node with 48 OpenMP and BLAS threads, as the default whole-node setting. For mid-size systems this turned out to be a poor configuration: tamoxifen B3LYP/def2-SVP took 1654 s at 48 threads but 519 s in the 24-thread reference run, with identical SCF iterations and energies. Over the complete 24-thread lane the same pattern holds: the 24-thread, one-BLAS-thread run was the faster lane in 120 of 127 cases; for the 65 cases that take at least 20 s on 48 threads it is faster by a median of 2.0×, and by up to 33× for the finite-difference benzene HF Hessian (4614 s against 141 s). A thread scan on the five representative cases (Table 9, Figure 10) found the cause: with 48 OpenMP threads each thread's BLAS calls spawn 48 more, oversubscribing the node. Setting BLAS threads to one restores the performance (tamoxifen 364 s at 48 threads, 307 s at 24 threads; vitamin C 24 s and 21 s against 55 s), pinning helps little, and for small molecules fewer threads are better still: benzene is fastest with 12 threads (4.0 s against 17.7 s with 48). For Taxol with PBE the spread is small (1594–1972 s). A second lane with 24 threads and one BLAS thread was therefore run for every case, and the fair CPU baseline of each case is the faster of the two lanes.

case	1	2	4	8	12	24	48	24/B1	48/B1	24 pin	48 pin
H_2O	4.54	4.30	4.13	4.12	4.11	4.58	**3.52**	–	–	–	–
benzene	15.3	9.06	6.07	4.33	**4.01**	7.80	17.7	–	–	–	–
vitamin C	–	–	–	–	22.3	23.5	55.3	**20.6**	24.0	23.5	43.2
tamoxifen	–	–	–	–	319.8	357.8	1663	**307.1**	363.8	322.2	946.6
Taxol PBE	–	–	–	–	1972	1604	1717	**1594**	1622	1604	1700

Table 9: CPU thread scan, wall time in seconds on one 48-core node. Columns: OpenMP threads with BLAS threads equal to OpenMP threads; 'B1' = BLAS threads set to 1; 'pin' = OpenMP threads pinned to cores. Bold: fastest for that case.

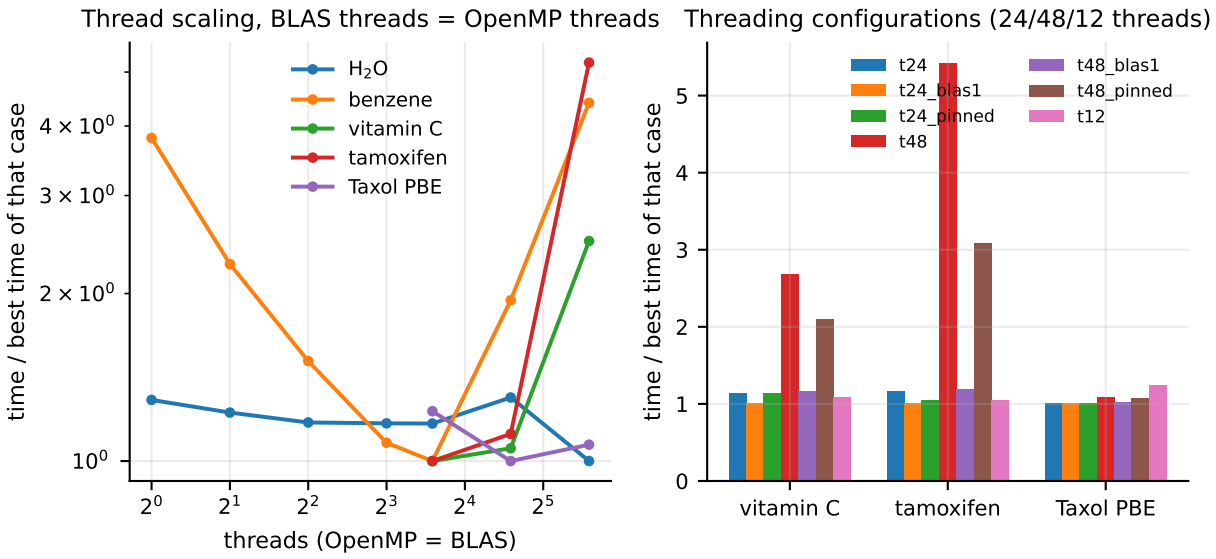


Figure 10: CPU thread scaling. Left: time relative to the best configuration against thread count (BLAS threads equal to OpenMP threads). Right: threading configurations for the three larger cases.

Table 10 and Figure 11 compare the CPU with the new GPU code. For the cases in which the CPU needs at least 20 s (41 on the A100, 41 on the V100, 38 on the P100-16) the median speed-up of the A100, V100 and P100-16 is 46, 36 and 23 against the best CPU lane, with maxima of 328, 331 and 197; the largest ratios belong to the finite-difference Hessians, which the CPU computes from many gradients. For the smallest molecules (a few seconds on the CPU) the GPU gains only 1–3× or nothing: about a dozen cases with sub-second to few-second CPU times (H_2O with HF, direct SCF, DF-MP2, CCSD, polarizability and shielding, and the formaldehyde wB97X TDA) are as fast or faster on the CPU, dominated by GPU start-up and by small kernels. The CPU baseline columns of the table state which lane was used.

case	AOs	CPU 48 thr (s)	CPU best (s)	A100 new (s)	V100 new (s)	P100-16 new (s)	speed-up A100
h2o-b3lyp-svp	24	3.16	0.51 (cpu24b1)	1.01	0.99	1.22	1
benzene-b3lyp-svp	114	17.7	3.67 (cpu24b1)	1.28	1.31	1.69	3
vitamin_c-b3lyp-svp	208	51.7	17.3 (cpu24b1)	2.34	2.18	3.21	7
inosine-b3lyp-svp	326	118.8	72.4 (cpu24b1)	3.69	3.98	6.55	20
tamoxifen-b3lyp-svp	537	1654	310.6 (cpu24b1)	6.49	7.80	12.0	48
tamoxifen-b3lyp-tzvp	1042	683.0	593.6 (cpu24b1)	12.8	18.2	26.0	46
taxol-pbe-svp	1123	1702	1604 (cpu24b1)	18.0	24.2	39.3	89
taxol-tpss-svp	1123	2504	2063 (cpu24b1)	26.4	38.1	60.6	78
vitamin_c-b3lyp-hessian	208	20730	5268 (cpu24b1)	20.1	20.9	37.8	262
inosine-b3lyp-tddft	326	949.1	790.2 (cpu24b1)	8.99	9.87	17.1	88
tamoxifen-b3lyp-iefpcm	537	1931	927.4 (cpu24b1)	8.57	10.4	16.1	108
vitamin_c-dfmp2	208	18.7	2.13 (cpu24b1)	1.11	1.17	1.59	2

Table 10: CPU wall time against the new GPU code. CPU 48 thr: whole 48-core node, default settings; CPU best: the faster of the available CPU lanes (configuration in brackets); speed-up = CPU best / A100.

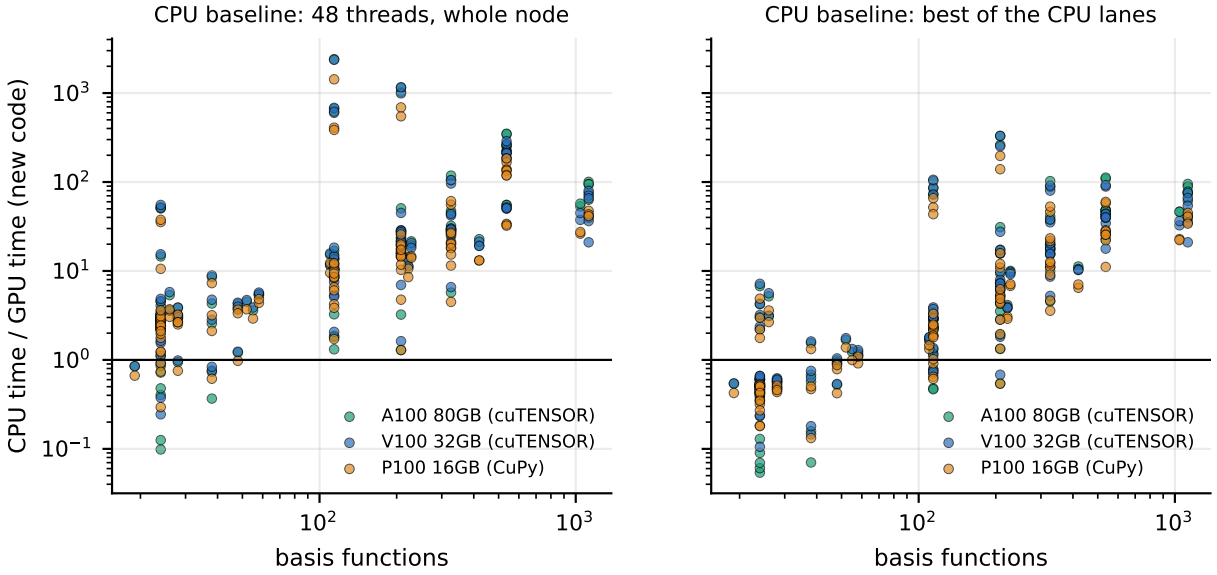


Figure 11: CPU time divided by new-GPU time against system size. Left: CPU baseline from the 48-thread whole-node lane only. Right: baseline from the faster of the available CPU lanes.

10 Memory and capacity

10.1 GPU memory of the workflow

The new code needs less GPU memory for every case that contains an exchange build and a gradient (Figure 12 and Table 11). Over the 55 cases with an original peak above 0.3 GB the median reduction is 2.1× (range 1.00–4.8×): tamoxifen B3LYP falls from 15.9 to 4.4 GB (3.7×), PCM and SMD cases 2.9–3.7×, tamoxifen in def2-TZVP from 24.3 to 11.6 GB (2.1×), and Taxol with the pure functionals LDA, PBE and TPSS 3.5–4.8× (from about 5 to 1–1.5 GB). Taxol with the hybrid functionals falls from 29.5 to 20.0 GB (HF, B3LYP; 1.5×) and, for wB97X, from 29.6 to 25.9 GB (1.1×). Hessians, TDA/TDDFT and DF-MP2 are unchanged (1.00×): the changes do not touch their code paths.

10.2 Storage tiers

The fitted integrals can be kept on the GPU, in pinned host memory, or (new code only) in a file read back in bounded blocks. Table 12 and Figure 13 give time and GPU memory for Taxol (which has a CPU reference and passes the accuracy gate in every tier for the new code) and for valinomycin (1542 basis functions, a capacity observation). Taxol on the V100 takes 59.3 s resident on the GPU, 71.7 s with the integrals in host memory (+21%) and 254.5 s from disk, while the gradient-phase GPU peak falls to 4.1 GB in every tier; the original code needs 29.6 GB for the gradient even with the integrals in host memory, and 54.9 GB on the A100. On the A100 the host tier costs +31% (46.4 s against 35.4 s) because the faster card hides less of the transfer. On the 12 and 16 GB P100 cards neither the original code nor the GPU-resident new code

case	AOs	original (GB)	new (GB)	ratio
`vitamin_c-b3lyp-svp`	208	1.24	0.50	2.50
`inosine-b3lyp-svp`	326	4.60	1.29	3.58
`tamoxifen-lda-svp`	537	2.42	0.65	3.74
`tamoxifen-b3lyp-svp`	537	15.89	4.35	3.66
`tamoxifen-wb97x-svp`	537	15.93	5.20	3.07
`tamoxifen-b3lyp-tzvp`	1042	24.30	11.57	2.10
`taxol-lda-svp`	1123	4.89	1.02	4.78
`taxol-pbe-svp`	1123	5.06	1.42	3.56
`taxol-tpss-svp`	1123	5.08	1.45	3.51
`taxol-hf-svp`	1123	29.53	19.97	1.48
`taxol-b3lyp-svp`	1123	29.33	20.04	1.46
`taxol-wb97x-svp`	1123	29.57	25.85	1.14
`tamoxifen-b3lyp-smd`	537	16.51	4.47	3.69
`vitamin_c-b3lyp-hessian`	208	2.57	2.57	1.00
`inosine-b3lyp-tddft`	326	5.58	5.58	1.00
`vitamin_c-dfmp2`	208	0.80	0.80	1.00

Table 11: Tracked GPU memory peak of the whole workflow on the V100 (memory-mode run; excludes the CuPy pool and native workspaces).

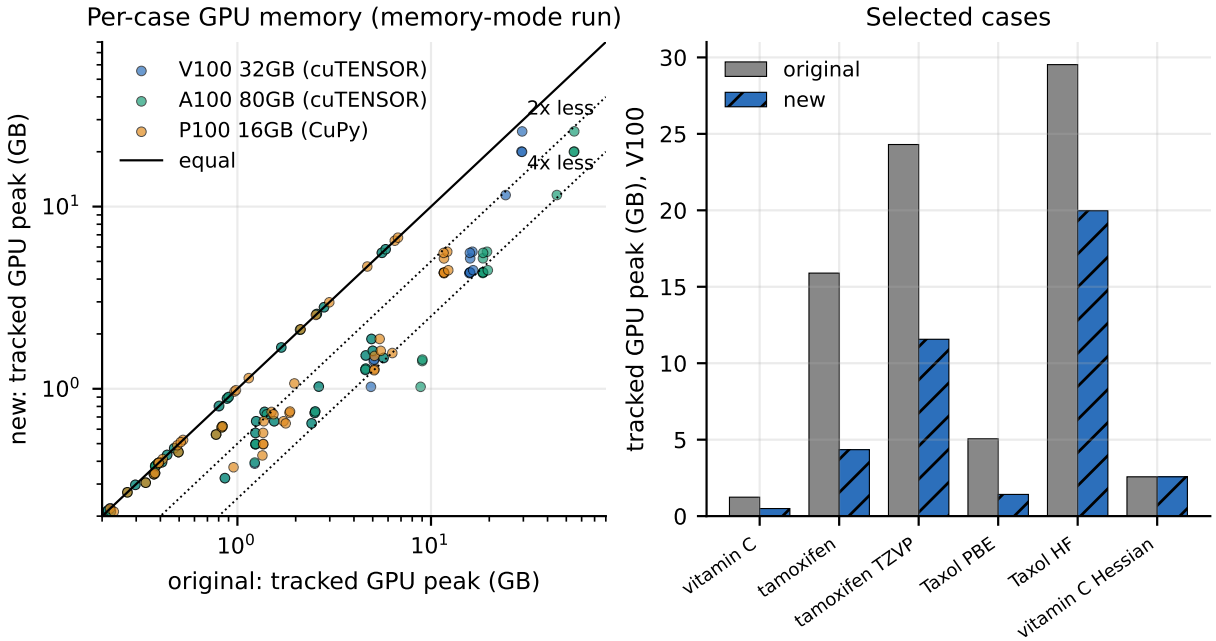


Figure 12: Tracked GPU memory peak of the new against the original code. Left: every case with an original peak above 0.2 GB on three strata (diagonal: equal; dotted: 2× and 4× less). Right: selected cases on the V100.

can start Taxol (the 15.5 GB fitted-integral tensor alone does not fit), but the new code with host memory completes it in 111.5 and 91.5 s using 4.7 GB of GPU memory. Valinomycin does not fit the 32 GB V100 in either code version when resident (31.5 GB tensor); with host memory the original code needs 75.5 GB of GPU memory for its gradient (A100) while the new code needs 8.7 GB, so valinomycin runs on both P100 cards (265 s on the 12 GB and 221 s on the 16 GB card). The disk tier is 3–6× slower than host memory; its times depend on the shared filesystem (Section 12).

system	card	version / tier	time (s)	GPU peak, gradient (GB)	host RSS (GB)
Taxol (1123 AOs)	V100 32GB	orig. GPU	59.1	29.6	–
	V100 32GB	orig. host	70.8	29.6	17
		new GPU	59.3	4.1	1
		new host	71.7	4.1	17
		new disk	254.5	4.1	3
	A100 80GB	orig. GPU	33.7	54.9	–
	A100 80GB	orig. host	43.8	54.9	17
		new GPU	35.4	4.1	1
		new host	46.4	4.1	17
		new disk	215.2†	6.6†	3
	P100 16GB	orig. GPU	out of memory	–	–
	P100 16GB	orig. host	out of memory	–	–
		new host	91.5	4.7	16
	P100 12GB	orig. GPU	out of memory	–	–
	P100 12GB	orig. host	out of memory	–	–
		new host	111.5	4.7	16
valinomycin (1542 AOs)	V100 32GB	orig. GPU	out of memory	–	–
	V100 32GB	orig. host	174.4	–	33
		new GPU	out of memory	–	–
		new host	185.0†	14.8†	33
		new disk	577.6†	14.8†	6
	A100 80GB	orig. GPU	79.4	–	1
	A100 80GB	orig. host	88.2	75.5	33
		new GPU	79.1†	14.8†	1
		new host	97.2	8.7	33
		new disk	524.4†	14.8†	6
	P100 16GB	orig. host	out of memory	–	–
		new host	220.9	8.7	32
	P100 12GB	orig. host	out of memory	–	–
		new host	264.9	8.7	32

Table 12: Storage tiers for the fitted integrals. Taxol has a CPU reference and passes the accuracy gate for the new code in every tier (the original code fails its gradient gate); valinomycin is a capacity observation. The original code has no disk tier. Rows marked † were run before the final gradient-memory fixes (their gradient peaks are 6.6 and 14.8 GB instead of 4.1 and 8.7 GB); all other new-code rows use the final code.

10.3 Capacity ladder

Table 13 and Figure 14 give the largest system each code version and tier completed with a B3LYP/def2-SVP energy and full-response gradient. The new code completes a cluster of 139 water molecules (417 atoms, 3336 basis functions) on the A100 in 17.9 min with host memory and 45.6 min from disk, against 300 atoms (2400 functions) for the original code; on the 32 GB V100 it reaches a 213-atom glycine chain (2154 functions) against 228 atoms (1824 functions) for the original with host memory, while resident on the GPU both versions stop at Taxol (1123 functions); on the two P100 cards it reaches valinomycin (168 atoms, 1542 functions) against tamoxifen (57 atoms, 537 functions) for the original, so a 12 GB card treats a system roughly three times larger; the 16 GB V100 reaches valinomycin with both versions (the original with host memory). These are capacity observations without a CPU reference.

What limits larger systems. Beyond these rungs the limit is not the fitted-integral tensor but a tensor of $N_{\mathrm{aux}}N_{\mathrm{occ}}^2$ elements (N_{aux} counted in Cartesian functions) that the analytic gradient holds on the GPU whichever storage tier is used. It is 2.7 GB for Taxol, 6.4 GB for valinomycin, 26.6 GB for the 100-water cluster, 50.1 GB for the 40-residue glycine chain, 71.4 GB for the 139-water cluster, and 97.3 GB for the 50-residue chain, which exceeds the 80 GB of the A100 and ends the ladder there. The storage tiers of

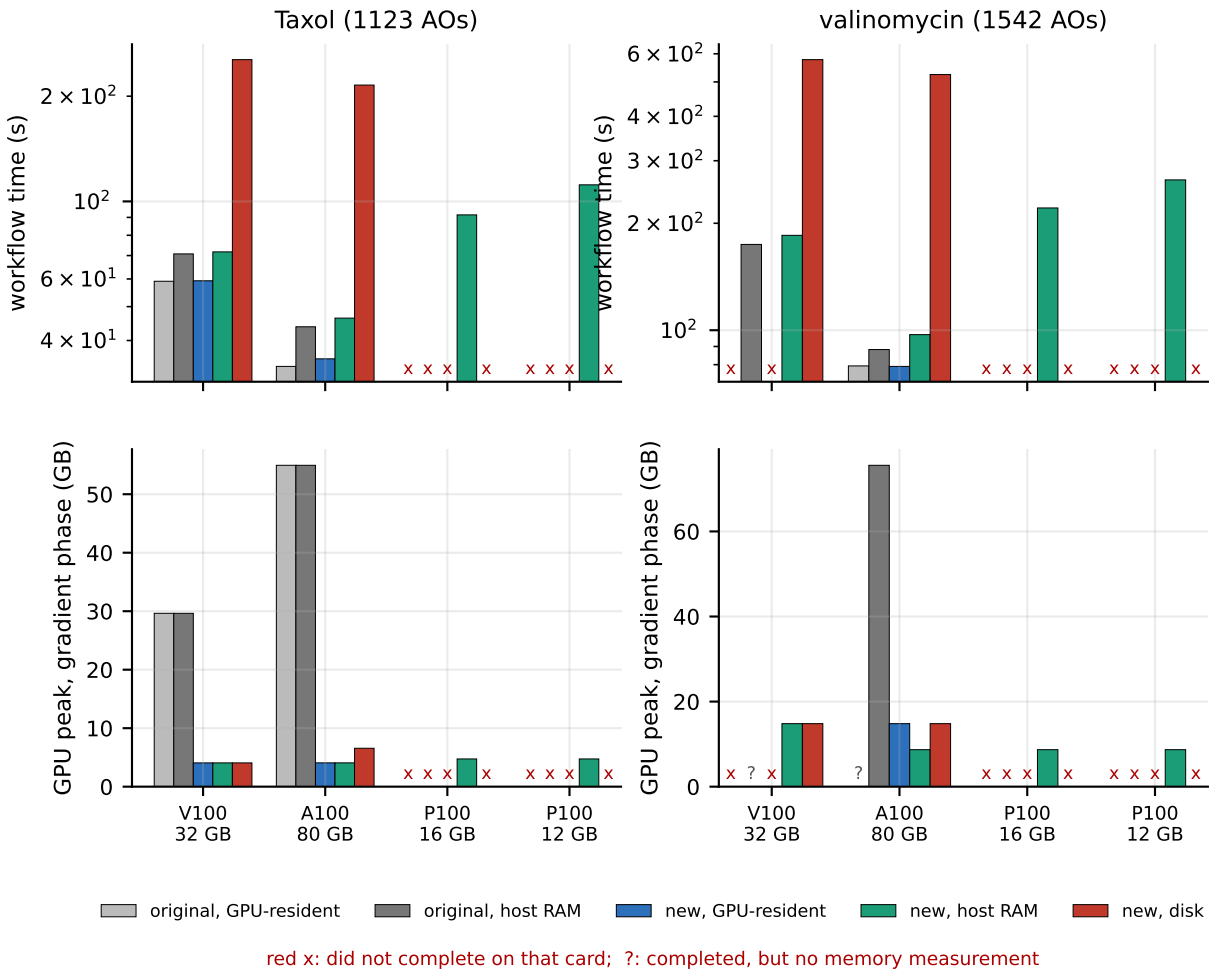


Figure 13: Time (top) and gradient-phase GPU memory (bottom) of Taxol (left) and valinomycin (right) by storage tier and card.

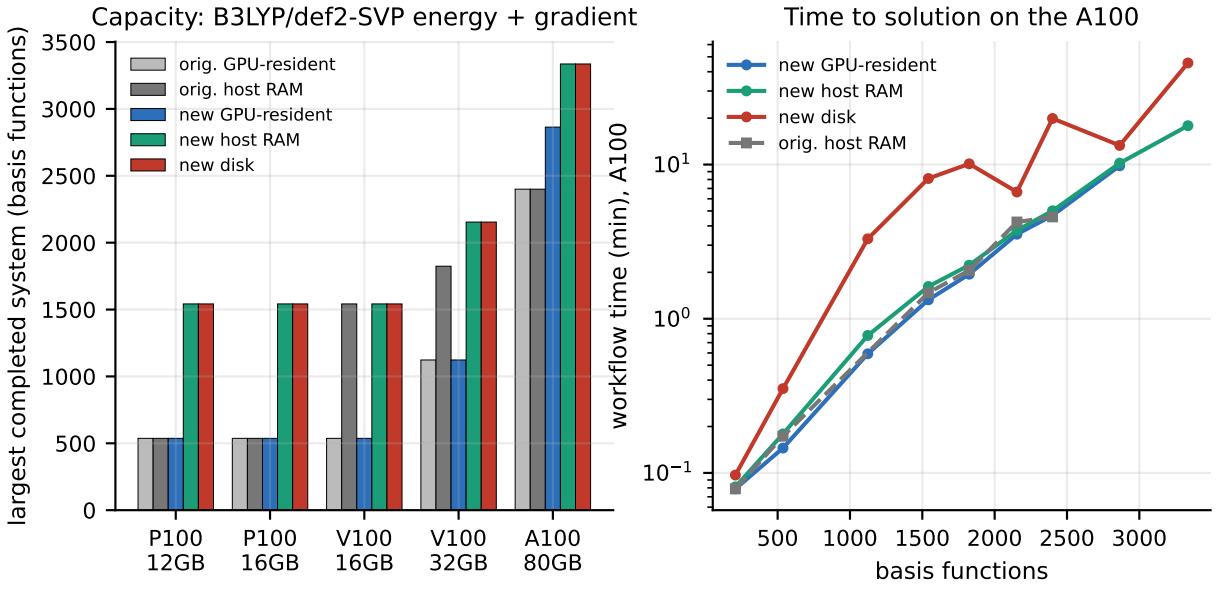


Figure 14: Left: largest system completed by card, code version and storage tier. Right: workflow time on the A100 against system size.

card	version / tier	largest system	atoms	AOs	time (min)
P100 12GB	orig. GPU	tamoxifen	57	537	0.39
	orig. host	tamoxifen	57	537	0.42
	new GPU	tamoxifen	57	537	0.22
	new host	valinomycin	168	1542	4.41
	new disk	valinomycin	168	1542	12.0
P100 16GB	orig. GPU	tamoxifen	57	537	0.27
	orig. host	tamoxifen	57	537	0.39
	new GPU	tamoxifen	57	537	0.20
	new host	valinomycin	168	1542	3.70
	new disk	valinomycin	168	1542	12.9
V100 16GB	orig. GPU	tamoxifen	57	537	0.13
	orig. host	valinomycin	168	1542	3.17
	new GPU	tamoxifen	57	537	0.14
	new host	valinomycin	168	1542	3.05
	new disk	valinomycin	168	1542	11.5
V100 32GB	orig. GPU	Taxol	113	1123	0.94
	orig. host	$(H_2O)_{76}$	228	1824	4.54
	new GPU	Taxol	113	1123	0.97
	new host	gly30	213	2154	8.99
	new disk	gly30	213	2154	11.7
A100 80GB	orig. GPU	$(H_2O)_{100}$	300	2400	4.29
	orig. host	$(H_2O)_{100}$	300	2400	4.58
	new GPU	gly40	283	2864	9.82
	new host	$(H_2O)_{139}$	417	3336	17.9
	new disk	$(H_2O)_{139}$	417	3336	45.6

Table 13: Largest system completed with B3LYP/def2-SVP energy and full-response gradient. These are capacity observations: no CPU reference exists, so only finite values and convergence are checked.

the fitted integrals are therefore exhausted at about 3300 basis functions on the largest card; going further requires the gradient to hold this tensor in host memory in blocks, which is not implemented.

11 Discussion

What the changes buy. (i) *Memory*: a median 2.1× and up to 4.8× less GPU memory for SCF energy-plus-gradient work with an exchange build, which together with the new disk tier and the host tier lets a 12 GB GPU treat systems of about 1500 basis functions and an 80 GB GPU systems of about 3300. (ii) *Speed on the fallback engine*: 1.2–1.8× for intermediate-size hybrid-functional jobs, up to 2.4×. (iii) *No loss of accuracy*: the worst qualified errors are the same or smaller than for the original code. The changes do *not* accelerate jobs that already use cuTENSOR, where the new code is on average 0–1% different from the original and 3–8% slower for hybrid functionals in mid-size cases, nor jobs whose code paths are not modified (Hessians, TDDFT, MP2, CCSD, properties, optimizations, periodic).

Relation to the earlier GB10 study. The earlier study reported 1.47× for tamoxifen and 1.95× for Taxol (Table 14). Here the same direction appears only without cuTENSOR and with smaller factors; with cuTENSOR it disappears. The GB10 had no cuTENSOR bindings, much lower double-precision throughput and unified memory, so its result describes cards and installations without a working cuTENSOR (for example consumer GPUs) rather than datacenter GPUs.

Recommendations. (1) Keep cuTENSOR as the default engine where available: without it the original code is slower by a median of 1.12–1.14× and by up to 2.57× (tamoxifen B3LYP/def2-TZVP on the A100), while the new code is slower by a median of 1.02–1.06× and at most 1.30×. (2) Treat the new code's value on such systems as memory and capacity, and on fallback-engine systems also speed. (3) The next memory limit is the gradient tensor; offloading it to host memory in auxiliary blocks would extend all tiers. (4) The remaining time is dominated by the exchange build near the double-precision ceiling and, for mid-size systems, by the XC quadrature (73% of the tamoxifen SCF on the A100), which is where further speed work would have to focus; neither start-up costs (Section 8) nor the SCF cycle count offer large gains.

12 Limitations and caveats

- **Timing noise and sharing.** Timings come from fresh processes on shared cluster nodes. GPU nodes were not exclusive during the main matrix, so the other GPU of a two-GPU node (and the filesystem) may have been in use; medians over five to nine samples bound this (the spread of the individual cases is visible in Figures 3 and 4), but ratios within a few percent of one should be read as "no difference". Disk-tier timings depend on the page cache and the shared filesystem: two alternating runs of identical code on one node differed by up to 12%, and single runs ranged from 189 to 254 s. No I/O performance claim is made.
- **Small systems.** Many core cases finish in one to a few seconds, where start-up costs, kernel launch latency and the CPU-side set-up dominate; ratios near one for these are expected and do not measure the numerical kernels.
- **Memory figures** are the peak of requested live CuPy storage in a separate instrumented process. They exclude the CuPy pool overhead, cuTENSOR and cuBLAS workspaces and native allocations, so the real device footprint is somewhat higher, and the P100-16 rows that report *resource limited* without a measurement are forecasts made by the scheduler before launch (they are labelled in the data).
- **Capacity runs** have no independent reference; they are checked only for finite values and SCF convergence. The largest system with an accuracy-qualified comparison to CPU PySCF is Taxol.
- **Registry amendments** (Section 2.7) change the numerical protocol relative to the original plan. In particular the GPU TDA calculations run at a tighter solver tolerance than the TDDFT calculations, so the TDA and TDDFT times must not be compared with each other; and the CPU Hessian timing is a finite-difference route, not PySCF's analytic Hessian.
- **Non-converged and ill-conditioned cases.** Four open-shell cases do not converge on CPU and are excluded; two of the three 48-thread CPU repetitions of NO/B3LYP differ from the 24-thread reference by 1.1×10^{-8} and 1.3×10^{-8} Eh (gate 10^{-8}) and are kept as accuracy failures; two of five benzene TDA

and TDDFT roots are flagged non-converged by the GPU solver at the declared tolerance for both code versions, although the excitation energies agree with CPU to 1.2×10^{-9} Eh.

- **The original code fails the CPU gradient gate on Taxol** for HF, B3LYP and wB97X (maximum component error about 2×10^{-5} Eh/Bohr), on all cards. Those cases have no equal-accuracy ratio.
- **Runner labelling.** Plain CUDA out-of-memory errors (`cudaErrorMemoryAllocation`) are recorded by the runner as *infrastructure failures*; they are genuine resource limits and are discussed as such. Pascal GPUs also fail CCSD with a native kernel launch error whose cause was not diagnosed.
- **Audit tolerance.** The suite's audit demands exact equality of recomputed comparison statistics; last-digit differences between processes (multi-threaded reductions) required a wrapper that compares floating-point values to a relative 10^{-9} and everything else exactly. Pre-launch admission rejections are accepted without a request file only when they carry their forecast.
- **Code-review findings left unchanged:** the 128 MiB default staging buffer (the campaign sets it explicitly), the GGA density shortcut (empirically within every gate, but its sum-of-squares form is no longer exactly non-negative), the `strict_grid_order` compatibility flag, the missing early guard for disk tensors in the old DF-MP2 module, and a duplicated staging-cap check.
- **CPU baseline.** The CPU is not optimized in this study. A 48-thread whole-node lane (the whole-node default) and a 24-thread, one-BLAS-thread lane are reported; the fair baseline for each case is the faster of the lanes. Other CPU settings (other thread counts, different BLAS) were not scanned for every case.

13 Conclusions

1. **The new GPU code is numerically equivalent to the original.** Across all qualified results, and across all job types and cards, the deviations from CPU PySCF are within the declared gates and not systematically different between the two code versions. The failures that remain are either the original code's own (Taxol gradients with hybrid functionals) or common to both versions and the shared native stack (wB97M-V, periodic MgO).
2. **Speed depends on the contraction engine.** With cuTENSOR the new code is no faster than the original (median ratio 1.00); without it, which is the case on the Pascal cards and on installations without a working cuTENSOR, it is faster by 1.2–1.8× for intermediate-size hybrid-functional calculations and by up to 2.4×. The original code is the one that suffers from the loss of cuTENSOR.
3. **Memory is the main gain.** A median 2.1× and up to 4.8× less GPU memory, plus a disk tier and a bounded build workspace, extend the reachable system size: valinomycin (1542 basis functions) on 12 and 16 GB cards, 3336 functions on the 80 GB A100. The price of the host tier is 20–30% of the run time and of the disk tier a factor of three to six.
4. **The next limit is the gradient.** A tensor of size $N_{\mathrm{aux}} N_{\mathrm{occ}}^2$ (97 GB for a 50-residue glycine chain) caps the system size on every card regardless of where the fitted integrals live; moving it to host memory in blocks is the most valuable next step.
5. **The exchange build is near the hardware ceiling.** The exchange matrix products run at 85–90% of the cuBLAS double-precision rate; the SCF-cycle count, starting guess, gradient workspace budget and start-up cost offer little. Remaining speed work should target the XC quadrature, which is most of the SCF time for mid-size systems on the A100.
6. **The CPU comparison must use a tuned baseline.** Using all 48 cores with default BLAS threading made CPU calculations slower than 24 threads with one BLAS thread in 120 of 127 cases (median 2.0× for the 65 cases above 20 s; up to 33× for the benzene HF Hessian); GPU speed-ups over the CPU should be quoted against the better of the two.

Data and code availability

The code changes, the benchmark suite, the campaign drivers, the audited summary data and the sources of this manuscript are available at `https://github.com/qshao/gpu4pyscf` (branch `improve-single-gpu`). The frozen campaign is identified by the SHA-256 digest given in Appendix A. The raw per-job records (about 66 GB) are not included in the repository and are available from the authors on request.

A Reproducibility

Frozen campaign. The 127-case campaign (`campaign1`) was frozen on 2026-10-20 with the SHA-256 digest of its `campaign.json` `231fea7a9e1722ad46b8c79eeb06789f0437118b8df3fa4976874d7459e4d380`. The file records the hashes of the G0 and G1 Python sources, the native libraries, all benchmark helper files, the registry and the hashed initial atomic densities; every job validates these identities before and after it runs, and each card works in a fork of the campaign whose `campaign.json` is byte-identical, so the CPU references are shared and one audit applies to all.

Layout. The code, benchmark suite and manuscript sources are in the `improve-single-gpu` branch of https://github.com/qshao/gpu4pyscf (commit `ac154f9` at submission). Code and data live in `benchmarks/single_gpu/qm_suite` of the repository (`cases.json`, `run.py`, `schedule.py`, `adapters.py`, `audit.py`); the cluster drivers are in `qm_suite/lcc`; the stage profiles, storage-tier and capacity runs are in `benchmarks/single_gpu/lcc_profile_results`; the manuscript is built from `reports/lcc_benchmark` (`collect_aux.py`, `make_figures.py`, `make_tables.py`, `report.tex`). The environment is a conda environment with Python 3.11, PySCF 2.14.0, CuPy 13.4.1, cuTENSOR 2.2.0 and the GCC 12 and CUDA 12.4 toolchains.

Commands. CPU references: a Slurm job array over the case list calling `run.py ... -variant CPU -mode reference`. GPU matrix: `schedule.py -campaign all -cases <list> -repetitions 5` on each card fork. CPU timing: `run.py ... -variant CPU -mode timing` per case and repetition on whole nodes. Capacity ladder: `lcc/capacity.py CARD`. Cross-stratum tables: `lcc/compare_variants.py`.

B Context: earlier single-GPU study on a GB10

Before the cluster campaign, the same branch was compared with the original code on one NVIDIA GB10 (unified memory, weak double-precision throughput, no cuTENSOR bindings so all contractions used the CuPy fallback). Table 14 reproduces that result. The GB10 study motivated two checks in the present report: whether the speed-up survives when cuTENSOR is available (Section 6), and whether the memory reduction matters on cards with far less memory (Section 10).

system	atoms	AOs	original (s)	new (s)	ratio	orig. GB	new GB
Vitamin C	20	208	5.49	5.28	1.04	1.4	0.5
Tamoxifen	57	537	70.5	48.1	1.47	21.1	4.3
Taxol	113	1123	1103	566.0	1.95	65.6	30.1

Table 14: Earlier study on one NVIDIA GB10 (unified memory, CuPy fallback contractions, B3LYP/def2-SVP, three timing repetitions); shown for context. The original code fails the CPU gradient gate on Taxol, so that ratio is not an equal-accuracy speed-up.

C Per-case results

Table 15: Per-case results. Columns A100 to P100-12: time ratio original/new ("nc": a solver or reference did not converge; "mem": resource limit; "acc": the original code fails the accuracy gate; "–": not run or not qualified). G1: new-code time on the V100; mem: V100 GPU memory original/new in GB; CPU: best CPU time.

case	AOs	A100	V100	A100c	V100c	P100-16	P100-12	G1 V100 (s)	mem (GB)	CPU (s)
c-acetate-b3lyp-svp	110	0.99	1.01	1.01	1.00	1.00	1.03	1.28	0.1/0.1	2.18
c-acetate-pbe-svp	110	1.00	0.97	1.00	0.98	0.99	1.01	1.37	0.1/0.1	2.45
c-benzene-b3lyp-svp	114	0.99	0.98	1.01	1.01	1.00	0.99	1.31	0.2/0.2	3.67
c-benzene-b3lyp-tzvp	222	1.01	1.02	1.02	1.01	1.03	1.07	1.59	0.9/0.3	6.26
c-benzene-hf-svp	114	1.02	1.03	1.02	1.04	1.01	1.06	0.77	0.2/0.1	0.86
c-benzene-lda-svp	114	0.99	0.98	1.02	1.01	1.00	1.01	1.20	0.1/0.1	2.79
c-benzene-pbe-svp	114	1.02	0.99	1.04	1.06	1.00	1.00	1.29	0.1/0.1	3.80
c-benzene-pbe-tzvp	222	1.00	0.99	1.00	1.01	0.99	1.02	1.55	0.2/0.2	5.96
c-benzene-tpss-svp	114	1.00	1.01	1.15	1.14	1.18	1.21	1.61	0.1/0.1	4.74
c-benzene-wb97x-svp	114	1.00	0.96	1.06	0.99	1.00	1.02	1.74	0.2/0.2	4.00
c-benzene_dimer-b3lyp-svp	228	0.98	0.98	0.99	1.01	1.01	1.01	2.08	1.5/0.7	19.9
c-benzene_dimer-pbe-svp	228	1.02	0.97	1.01	0.99	1.02	1.00	1.96	0.5/0.4	18.1
c-distorted_h2o-b3lyp-svp	24	0.98	1.00	1.02	1.00	1.01	1.00	1.03	0.0/0.0	0.51
c-h2o-b3lyp-pvdz	24	1.01	0.95	1.00	0.98	0.99	0.97	1.06	0.0/0.0	0.49
c-h2o-b3lyp-svp	24	1.01	1.04	1.01	1.00	0.98	1.04	0.99	0.0/0.0	0.51
c-h2o-hf-svp	24	1.04	1.04	1.03	1.01	0.99	1.01	0.71	0.0/0.0	0.38
c-h2o-lda-svp	24	1.01	1.03	0.98	0.95	1.00	1.05	1.00	0.0/0.0	0.56
c-h2o-pbe-pvdz	24	1.01	1.01	1.01	1.03	0.97	1.00	0.98	0.0/0.0	0.60
c-h2o-pbe-svp	24	1.00	1.03	0.99	1.01	1.00	1.08	0.97	0.0/0.0	0.59
c-h2o-tpss-svp	24	1.02	1.01	1.08	1.05	1.07	1.16	1.02	0.0/0.0	0.67
c-h2o-wb97x-svp	24	0.99	0.98	0.99	1.01	1.01	1.02	1.16	0.0/0.0	0.52
c-inosine-b3lyp-svp	326	1.01	0.99	1.02	1.05	1.05	1.09	3.98	4.6/1.3	72.4
c-inosine-hf-svp	326	0.94	0.95	0.95	1.03	1.06	1.13	2.00	4.6/1.3	17.3
c-inosine-lda-svp	326	1.01	0.99	1.01	1.00	1.00	1.00	3.11	0.8/0.6	48.5
c-inosine-pbe-svp	326	1.03	1.02	1.03	1.02	1.02	1.03	3.81	0.8/0.6	68.2
c-inosine-tpss-svp	326	1.00	1.00	1.24	1.24	1.37	1.53	5.22	0.8/0.6	106.8
c-inosine-wb97x-svp	326	0.99	0.98	0.99	1.02	1.04	1.08	4.97	4.6/1.5	77.3
c-methylammonium-b3lyp-svp	58	1.02	0.98	1.01	0.98	1.00	1.04	1.19	0.1/0.1	1.37
c-methylammonium-pbe-svp	58	0.99	0.98	0.99	1.01	1.01	1.01	1.22	0.1/0.1	1.57
c-no-b3lyp-svp	28	1.01	0.98	acc	acc	1.02	0.98	1.11	0.0/0.0	0.63
c-no-pbe-svp	28	nc	nc	nc	0.98	–	nc	1.07	0.0/0.0	–
c-o2-b3lyp-svp	28	0.98	1.01	1.00	1.00	1.01	0.97	1.01	0.0/0.0	0.56
c-o2-pbe-svp	28	1.01	1.02	1.02	0.99	1.00	0.97	1.01	0.0/0.0	0.63
c-oh-b3lyp-svp	19	–	–	–	nc	–	–	–	–	–
c-oh-pbe-svp	19	–	–	–	–	nc	–	–	–	–
c-stretched_n2-b3lyp-svp	28	1.00	0.98	0.97	1.00	1.03	1.00	0.96	0.0/0.0	0.47
c-tamoxifen-b3lyp-svp	537	0.96	0.96	1.31	1.34	1.39	1.77	7.80	15.9/4.3	310.6
c-tamoxifen-b3lyp-tzvp	1042	0.92	0.96	2.39	1.76	–	–	18.2	24.3/11.6	593.6
c-tamoxifen-hf-svp	537	0.88	0.91	2.34	1.75	1.63	2.26	4.61	15.8/4.3	82.2
c-tamoxifen-lda-svp	537	1.00	0.99	1.01	1.00	1.00	1.00	4.81	2.4/0.6	201.4
c-tamoxifen-pbe-svp	537	1.01	1.01	1.03	1.02	1.02	1.04	5.70	2.5/0.7	251.4
c-tamoxifen-pbe-tzvp	1042	1.01	1.00	1.00	1.00	1.01	1.01	10.8	2.6/1.0	392.7
c-tamoxifen-tpss-svp	537	1.01	1.00	1.24	1.24	1.36	1.43	8.72	2.5/0.8	381.9
c-tamoxifen-wb97x-svp	537	0.94	0.94	1.48	1.27	1.32	1.64	10.8	15.9/5.2	371.2
c-taxol-b3lyp-svp	1123	acc	acc	acc	–	mem	mem	58.8	29.3/20.0	2472
c-taxol-hf-svp	1123	acc	acc	acc	–	mem	mem	47.5	29.5/20.0	998.5
c-taxol-lda-svp	1123	1.01	1.00	1.00	1.00	–	1.01	18.3	4.9/1.0	1377
c-taxol-pbe-svp	1123	1.03	1.09	1.03	1.09	–	1.09	24.2	5.1/1.4	1604
c-taxol-tpss-svp	1123	1.00	1.00	1.20	1.05	–	1.21	38.1	5.1/1.4	2063

case	AOs	A100	V100	A100c	V100c	P100-16	P100-12	G1 V100 (s)	mem (GB)	CPU (s)
c-taxol-wb97x-svp	1123	acc	acc	acc	–	mem	mem	85.1	29.6/25.9	3006
c-vitamin_c-b3lyp-svp	208	1.04	1.01	1.03	1.02	1.02	1.03	2.18	1.2/0.5	17.3
c-vitamin_c-b3lyp-tzvp	420	1.00	0.99	1.09	1.14	1.12	1.19	3.24	5.7/1.5	34.0
c-vitamin_c-hf-svp	208	0.89	0.99	1.02	1.00	1.03	1.02	1.08	1.2/0.4	4.62
c-vitamin_c-lda-svp	208	0.99	0.99	1.00	0.99	0.99	1.00	2.03	0.3/0.3	12.3
c-vitamin_c-pbe-svp	208	1.03	1.01	1.01	1.00	1.03	0.98	2.33	0.4/0.3	16.8
c-vitamin_c-pbe-tzvp	420	1.00	1.01	1.00	1.01	1.00	0.99	3.19	0.4/0.4	32.7
c-vitamin_c-tpss-svp	208	1.00	0.99	1.22	1.22	1.33	1.54	3.14	0.4/0.3	20.9
c-vitamin_c-wb97x-svp	208	1.01	1.01	1.02	1.00	1.02	1.02	2.66	1.2/0.6	19.4
c-water_dimer-b3lyp-svp	48	0.99	1.01	1.00	1.01	0.99	0.99	1.12	0.1/0.1	1.05
c-water_dimer-pbe-svp	48	1.00	1.02	1.00	0.99	1.02	1.05	1.16	0.1/0.1	1.21
direct-benzene-b3lyp	114	1.00	1.00	0.99	0.99	1.00	1.00	2.13	0.2/0.2	6.45
direct-benzene-hf	114	1.00	1.01	0.99	0.99	0.99	0.98	1.61	0.2/0.2	3.66
direct-benzene-pbe	114	1.00	1.03	1.00	0.98	0.99	1.02	1.62	0.2/0.2	6.30
direct-h2o-b3lyp	24	1.00	1.02	1.00	0.98	1.01	0.98	1.41	0.2/0.2	0.55
direct-h2o-hf	24	1.00	0.99	1.00	1.03	0.97	1.02	1.10	0.2/0.2	0.38
direct-h2o-pbe	24	0.99	1.05	0.98	1.01	1.01	1.01	0.97	0.2/0.2	0.50
e-benzene-b3lyp-hessian	114	1.00	1.00	1.04	1.03	1.04	1.02	8.07	0.4/0.4	699.3
e-benzene-b3lyp-tda	114	nc	nc	nc	nc	nc	nc	–	0.4/0.4	66.7
e-benzene-b3lyp-tddft	114	nc	nc	nc	nc	nc	nc	–	0.9/0.9	32.9
e-benzene-dfmp2	114	0.96	1.02	1.02	1.03	1.08	1.02	0.75	0.2/0.2	0.58
e-benzene-hf-hessian	114	1.01	1.02	1.04	1.01	1.04	1.01	1.95	0.3/0.3	140.7
e-benzene-optimization	114	1.00	1.01	1.02	1.05	1.02	1.01	5.15	0.2/0.2	19.2
e-benzene-pbe-hessian	114	1.00	1.01	1.02	1.03	1.04	1.03	7.71	0.5/0.5	816.1
e-benzene-polarizability	114	1.00	1.01	0.99	0.98	1.01	0.99	2.90	0.1/0.1	3.94
e-benzene-shielding	114	1.00	0.99	1.00	1.00	1.01	1.01	7.51	0.2/0.2	5.52
e-benzene-wb97mv	114	acc	acc	acc	acc	acc	acc	–	0.2/0.2	55.6
e-benzene-wb97x-tda	114	nc	nc	nc	nc	nc	nc	–	0.4/0.4	159.9
e-benzene-wb97x-tddft	114	nc	nc	nc	nc	nc	nc	–	0.9/0.9	62.7
e-ch3i-pbe-ecp	55	1.02	0.99	1.01	1.02	1.01	1.01	1.80	0.1/0.1	2.38
e-formaldehyde-b3lyp-tda	38	1.01	1.01	1.00	1.00	1.01	0.99	2.91	0.1/0.1	1.83
e-formaldehyde-b3lyp-tddft	38	0.99	1.00	1.01	0.99	0.99	1.01	1.52	0.1/0.1	2.46
e-formaldehyde-ccsd	38	0.91	1.00	1.00	0.91	mem	mem	8.38	0.1/0.1	1.22
e-formaldehyde-optimization	38	1.00	1.01	1.01	1.02	1.00	1.02	4.23	0.1/0.1	3.18
e-formaldehyde-wb97x-tda	38	0.99	0.99	0.99	1.00	1.01	1.01	10.9	0.1/0.1	1.96
e-formaldehyde-wb97x-tddft	38	nc	nc	nc	nc	nc	nc	–	0.1/0.1	2.74
e-h2o-b3lyp-d3bj	24	1.02	0.97	1.01	1.04	1.02	0.92	0.98	0.0/0.0	0.49
e-h2o-b3lyp-d4	24	0.98	1.00	0.99	0.98	1.01	0.96	1.00	0.0/0.0	0.54
e-h2o-b3lyp-hessian	24	1.00	0.98	1.04	1.02	0.99	1.00	2.24	0.1/0.1	9.57
e-h2o-b3lyp-iefpcm	24	1.00	1.00	1.00	0.97	1.00	1.03	1.25	0.0/0.0	0.79
e-h2o-b3lyp-smd	24	0.99	1.03	1.00	0.95	0.97	1.01	1.23	0.0/0.0	0.81
e-h2o-ccsd	24	1.00	1.00	1.01	1.01	mem	mem	6.95	0.1/0.1	0.73
e-h2o-dfmp2	24	0.98	1.01	1.01	1.04	1.01	0.90	0.64	0.0/0.0	0.34
e-h2o-hf-hessian	24	1.00	1.01	1.01	1.01	1.02	1.04	1.29	0.0/0.0	4.14
e-h2o-optimization	24	1.00	1.02	1.00	1.00	1.01	1.02	2.74	0.0/0.0	1.40
e-h2o-pbe-hessian	24	1.01	1.03	1.01	1.02	1.01	1.00	2.14	0.1/0.1	15.3
e-h2o-polarizability	24	1.00	1.01	1.01	0.99	0.99	0.95	1.65	0.1/0.1	0.39
e-h2o-shielding	24	1.00	1.01	1.00	1.02	1.01	0.99	1.74	0.1/0.1	0.41
e-h2o-wb97mv	24	1.00	1.00	1.03	1.05	1.06	0.94	1.56	0.0/0.0	3.69
e-i2-pbe-ecp	52	1.00	1.00	1.02	0.99	1.00	1.00	1.38	0.1/0.1	2.42
e-inosine-b3lyp-iefpcm	326	0.99	0.99	1.01	1.03	1.04	1.06	5.30	4.9/1.9	186.7
e-inosine-b3lyp-smd	326	1.00	0.99	1.01	1.03	1.04	1.07	5.26	5.0/1.6	199.2
e-inosine-b3lyp-tda	326	1.00	1.01	1.01	1.01	1.01	1.02	131.5	1.7/1.7	688.8
e-inosine-b3lyp-tddft	326	1.00	1.01	1.01	1.03	1.07	1.05	9.87	5.6/5.6	790.2
e-inosine-wb97x-tda	326	nc	nc	nc	nc	nc	nc	–	2.1/2.1	889.1
e-inosine-wb97x-tddft	326	1.00	1.00	1.01	1.03	1.06	1.05	15.6	5.8/5.8	1421

case	AOs	A100	V100	A100c	V100c	P100-16	P100-12	G1 V100 (s)	mem (GB)	CPU (s)
e-no-dfump2	28	1.00	1.01	1.03	1.03	1.00	0.94	0.71	0.0/0.0	0.43
e-oh-dfump2	19	0.97	0.99	0.99	1.03	1.01	1.07	0.69	0.0/0.0	0.38
e-tamoxifen-b3lyp-d3bj	537	0.96	0.96	1.31	1.34	1.38	1.81	7.80	15.9/4.3	308.9
e-tamoxifen-b3lyp-d4	537	0.95	0.96	1.31	1.34	1.37	1.77	7.73	15.9/4.3	310.3
e-tamoxifen-b3lyp-iefpcm	537	0.97	0.97	1.20	1.23	1.26	1.52	10.4	16.4/5.7	927.4
e-tamoxifen-b3lyp-smd	537	0.97	0.98	1.20	1.23	1.25	1.53	10.4	16.5/4.5	954.7
e-tamoxifen-wb97mv	537	acc	acc	acc	acc	acc	acc	–	15.9/5.6	2768
e-vitamin_c-b3lyp-hessian	208	1.00	0.99	1.04	1.04	1.04	1.04	20.9	2.6/2.6	5268
e-vitamin_c-b3lyp-iefpcm	208	1.02	1.00	1.02	1.01	1.02	1.02	2.91	1.4/0.7	46.2
e-vitamin_c-b3lyp-smd	208	1.01	1.00	1.02	1.01	1.01	1.04	2.93	1.4/0.7	50.3
e-vitamin_c-dfmp2	208	1.01	0.97	1.06	1.03	1.03	0.91	1.17	0.8/0.8	2.13
e-vitamin_c-optimization	208	1.00	1.00	1.03	1.02	1.03	1.04	48.6	1.2/0.5	479.3
e-vitamin_c-pbe-hessian	208	1.01	1.00	1.03	1.04	1.02	1.04	18.7	2.8/2.8	6185
e-vitamin_c-polarizability	208	1.00	1.01	1.01	1.00	1.00	1.00	7.21	0.2/0.2	20.6
e-vitamin_c-shielding	208	1.00	1.00	1.00	1.01	1.01	1.01	28.0	0.3/0.3	19.1
e-vitamin_c-wb97mv	208	acc	acc	acc	acc	acc	acc	–	1.3/0.7	239.3
e-water_dimer-dfmp2	48	0.98	1.01	1.00	1.04	1.00	0.95	0.67	0.0/0.0	0.36
pbc-mgo-gamma	27	acc	acc	acc	acc	acc	acc	–	0.0/0.0	3.57
pbc-si-gamma	26	0.97	1.00	1.00	1.02	1.01	1.00	0.94	0.0/0.0	2.92
pbc-si-k222	26	0.98	1.05	1.01	1.03	1.04	1.04	1.40	0.2/0.2	7.84
pbc-si-supercell-gamma	208	1.00	1.02	1.04	1.03	1.00	0.99	2.12	0.1/0.1	58.6
probe-oh-wb97x-svp	19	–	–	–	–	–	–	–	–	–

References

[1] Q. Sun, T. C. Berkelbach, N. S. Blunt, G. H. Booth, S. Guo, Z. Li, J. Liu, J. D. McClain, E. R. Sayfutyarova, S. Sharma, S. Wouters, G. K.-L. Chan, PySCF: the Python-based simulations of chemistry framework, *WIREs Comput. Mol. Sci.* **8**, e1340 (2018).

[2] Q. Sun, X. Zhang, S. Banerjee, P. Bao, M. Barbry, N. S. Blunt, *et al.*, Recent developments in the PySCF program package, *J. Chem. Phys.* **153**, 024109 (2020).

[3] R. Li, Q. Sun, X. Zhang, G. K.-L. Chan, Introducing GPU-acceleration into the Python-based Simulations of Chemistry Framework, arXiv:2407.09700 (2024).

[4] X. Wu, Q. Sun, Z. Pu, T. Zheng, W. Ma, W. Yan, X. Yu, Z. Wu, M. Huo, X. Li, W. Ren, S. Gong, Y. Zhang, W. Gao, Enhancing GPU-acceleration in the Python-based Simulations of Chemistry Framework, arXiv:2404.09452 (2024).

[5] J. L. Whitten, Coulombic potential energy integrals and approximations, *J. Chem. Phys.* **58**, 4496 (1973).

[6] B. I. Dunlap, J. W. D. Connolly, J. R. Sabin, On some approximations in applications of $X\alpha$ theory, *J. Chem. Phys.* **71**, 3396 (1979).

[7] O. Vahtras, J. Almlöf, M. W. Feyereisen, Integral approximations for LCAO-SCF calculations, *Chem. Phys. Lett.* **213**, 514 (1993).

[8] F. Weigend, R. Ahlrichs, Balanced basis sets of split valence, triple zeta valence and quadruple zeta valence quality for H to Rn, *Phys. Chem. Chem. Phys.* **7**, 3297 (2005).

[9] F. Weigend, Accurate Coulomb-fitting basis sets for H to Rn, *Phys. Chem. Chem. Phys.* **8**, 1057 (2006).

[10] S. Lehtola, C. Steigemann, M. J. T. Oliveira, M. A. L. Marques, Recent developments in Libxc — a comprehensive library of functionals for density functional theory, *SoftwareX* **7**, 1 (2018).

[11] R. Okuta, Y. Unno, D. Nishino, S. Hido, C. Loomis, CuPy: a NumPy-compatible library for NVIDIA GPU calculations, in *Proc. Workshop on Machine Learning Systems (LearningSys) at NIPS* (2017).

[12] NVIDIA Corporation, cuTENSOR: a high-performance CUDA library for tensor primitives, https://developer.nvidia.com/cutensor.

[13] N. Mardirossian, M. Head-Gordon, ωB97M-V: a combinatorially optimized, range-separated hybrid, meta-GGA density functional with VV10 nonlocal correlation, *J. Chem. Phys.* **144**, 214110 (2016).

[14] S. Grimme, S. Ehrlich, L. Goerigk, Effect of the damping function in dispersion corrected density functional theory, *J. Comput. Chem.* **32**, 1456 (2011).

[15] E. Caldeweyher, S. Ehlert, A. Hansen, H. Neugebauer, S. Spicher, C. Bannwarth, S. Grimme, A generally applicable atomic-charge dependent London dispersion correction, *J. Chem. Phys.* **150**, 154122 (2019).